

基本情報技術者試験 科目B対策

# 擬似言語とAIで学ぶ アルゴリズム入門

はじめての変数から、配列のトレースまで

「答えを聞く」のではなく、  
「分からない場所を見つける」ためにAIを使おう。

基本情報技術者試験 科目B対策教材

# はじめに

---

この教材は、基本情報技術者試験の科目Bで出題されるアルゴリズムとプログラミングを、プログラミング未経験者が擬似言語から学ぶための入門書です。

科目Bの問題を解くために、難しいプログラム言語を最初から覚える必要はありません。まず必要なのは、次の三つの力です。

1. 擬似言語を上から順番に読む力
2. 変数や配列の値がどう変化するかを追う力
3. 分からない箇所を言葉にして、質問する力

本書では、処理の基本となる順次・選択・繰返しを擬似言語で学びます。各演習の解説には、処理の進み方を矢印で示すフローチャートも掲載します。コードの一行と図の一つの処理を対応させて読みましょう。

また、本書には生成AIへ質問するための例文を掲載しています。AIに正解を直接聞くのではなく、考え方の整理、ヒント、トレースの確認、間違いの発見に利用します。

## 本書の到達目標

- 変数・代入・配列を説明できる
- 選択処理と繰返し処理を読める
- 短い擬似言語をトレースできる
- 合計・件数・最大値を求める処理を理解できる
- AIを使って、自分で学習を続けられる

## この教材の使い方

---

第0章は、全員が最初に学ぶ「AIの安全で有効な活用」です。第1～8章には各6問(★を2問、★★を2問、★★★を2問)、合計48問を用意しています。既存の問題番号は変更していないため、英字の順番と難易度の順番が異なる章があります。

| 難易度    | 取り組み方                  | 一問の目安  |
|--------|------------------------|--------|
| ★☆☆ 基礎 | 一つの処理・値・条件を確認する        | 3～5分   |
| ★★☆ 応用 | 複数の処理を組み合わせ、途中の値を説明する  | 5～10分  |
| ★★★ 実践 | 仕様や利用場面を読み、境界や処理順も検証する | 10～20分 |

時間は目安です。講師が受講者の理解度に合わせて選びます。早く終わった人は上の難易度へ進み、条件や入力を変えても説明できるか確かめます。★★★は本番と同じ難しさを保証する表示ではなく、各章の知識を実践的に使う研修用問題です。公開問題へ進むための読み方と参照先も掲載しています。

各節は、次の順番で進みます。

|                                          |                                             |                                         |                                            |                                         |
|------------------------------------------|---------------------------------------------|-----------------------------------------|--------------------------------------------|-----------------------------------------|
| <b>1</b><br><b>概念を読む</b><br>何をする仕組みか理解する | <b>2</b><br><b>処理をたどる</b><br>値が変わる順番を手で記録する | <b>3</b><br><b>自力で解く</b><br>最初からAIに聞かない | <b>4</b><br><b>AIに質問する</b><br>分からない箇所だけ尋ねる | <b>5</b><br><b>解き直す</b><br>AIを閉じてもう一度解く |
|------------------------------------------|---------------------------------------------|-----------------------------------------|--------------------------------------------|-----------------------------------------|

## 大切なルール

AIの説明は、いつも正しいとは限りません。本書の解説や公式資料と照合し、自分で値を追って確認してください。「AIが言ったから正しい」ではなく、「自分でも同じ結果を確認できたから正しい」と判断します。

## 目次

|                       |     |
|-----------------------|-----|
| 第0章 AIを学習相手にする .....  | 5   |
| 第1章 アルゴリズムと擬似言語 ..... | 11  |
| 第2章 変数・データ型・代入 .....  | 19  |
| 第3章 順次処理 .....        | 27  |
| 第4章 選択処理 .....        | 33  |
| 第5章 繰返し処理 .....       | 42  |
| 第6章 配列 .....          | 51  |
| 第7章 トレースの技術 .....     | 59  |
| 第8章 総合演習 .....        | 67  |
| 解答・解説 .....           | 72  |
| 研修後の学習ロードマップ .....    | 121 |

# 第0章 AIを学習相手にする

## 最初に確認:AIを安全に、有効に使う

この章は演習より先に学びます。AIを使い慣れていても、安全のルールと学習に役立つ質問方法を確認してください。

| 確認すること    | 研修での行動                                                          |
|-----------|-----------------------------------------------------------------|
| 個人情報・機密情報 | 氏名、連絡先、顧客情報、社内資料、パスワードは入力しません。必要な例は架空の名前・数値に置き換えます。             |
| 利用する場所    | 講師が指定した共有ノートブックを使い、会社・研修の利用ルールを守ります。リンクを研修外へ転送しません。             |
| 添付する権限    | 他の参考書を使う場合は、利用条件を確認します。購入した本でも、全ページのアップロードや他人への共有が認められるとは限りません。 |
| 回答の正しさ    | AIの説明はもっともらしくても誤ることがあります。参照元を開き、自分のトレースと教材の解説で確かめます。            |
| 学習の目的     | まず自分で考え、ヒントを一つずつ求めます。最後はAIを閉じて解き直します。                           |

「共有先が限られている」とことと「秘密情報を入力してよい」ことは別です。入力してよいか迷ったら、送信する前に講師へ確認してください。誤って送った場合は追加送信や拡散を止め、研修の連絡ルールに従って講師へ報告します。

**安全確認の例:**「顧客Aさんの実際の購入履歴」ではなく、「架空の購入額が1000円の場合」として質問します。学習に必要なのない実データは渡しません。

### 0.1 AIは「正解を出す機械」ではない

生成AIへ問題をそのまま入力し、「答えを教えて」と頼めば、短時間で答えらしきものが表示されます。しかし、それだけでは自分で問題を解く力は身に付きません。

本書では、AIを次の四つの役割で使います。

| AIの役割 | 何をしてもらうか      | 使用する場面       |
|-------|---------------|--------------|
| 整理役   | 入力・処理・出力を分ける  | 問題文の意味が分からない |
| ヒント役  | 次に見るべき場所を示す   | 解き始められない     |
| 点検役   | 最初に間違えた箇所を示す  | 自分の考えに自信がない  |
| 練習相手  | 難易度を調整した類題を作る | もう一問練習したい    |

### 避けたい質問

問題番号を示さずに「答えを教えてください」とだけ頼むことや、考える前に「完成した擬似言語を作ってください」と頼むこと

正解だけを受け取ると、どこで考えられなくなったのかが分かりません。

### 学習につながる質問

「答えはまだ示さず、最初に確認すべき変数を一つ教えてください」

「私のトレース表で、最初に間違っている行だけを指摘してください」

## 0.2 AIに質問する前の30秒

質問を入力する前に、次の三点を書き出します。

| AIへ聞く前の整理メモ |                |
|-------------|----------------|
| 1 分かっていること  | ここに書く<br>----- |
| 2 分からないこと   | ここに書く<br>----- |
| 3 自分で試したこと  | ここに書く<br>----- |

例えば、次のように書きます。

#### 記入例

- 1 分かっていること  
totalが合計を保存する変数であることは分かります。
- 2 分からないこと  
`total ← total + A[i]` の右辺で、どの値を使うのか分かりません。
- 3 自分で試したこと  
iが1の場合までは表に書きました。

ここまで書くと、AIへの質問が具体的になります。また、自分がどこまで理解できているかも見えるようになります。

## 0.3 研修中:共有ノートブックへ問題番号で質問する

今回の研修では、講師が本書のPDFを読み込ませたGemini Notebook(NotebookLM)のリンクを共有します。受講者が問題文をコピーして貼り付けたり、PDFを再アップロードしたりする必要はありません。講師から届いたリンクで対象ノートブックを開き、そのチャット欄に質問します。

1. 共有リンクを開き、研修用のノートブックであることを確認する。
2. 「演習2-B」のように、章番号と問題の英字を指定する。
3. 自分が分かっていること、試したこと、分からない点を書く。
4. 「答えはまだ示さず、ヒントを一つ」と頼む。
5. 回答の参照元と問題の題名を確認し、別の問題について答えていないか確かめる。

本教材の演習2-B「値を交換する」について質問です。xにyを入れるところは分かりますが、workが必要な理由が分かりません。答えや完成コードはまだ示さず、最初の値がどこに残るかを考えるヒントを一つください。

問題が見つからないと言われたら、問題番号に教材名・題名を添えて指定し直します。それでも見つからない場合は、講師に共有先と登録PDFの確認を依頼してください。通常のGeminiの新しいチャットを開くだけでは、この研修のPDFを参照できるとは限りません。

**講師の準備:** 修正版PDFを登録し、共有対象と閲覧権限を確認します。受講者と同じ利用条件で「演習2-B」と「演習8-F」の問題を正しく特定できるか、出典表示も含めて試してください。本PDFには解答も含まれるため、「答えをまだ示さないで」は学習上のお願いであり、解答の閲覧を技術的に禁止する仕組みではありません。

## 0.4 研修後:別の参考書の問題をAIへ伝える

共有ノートブックに登録されていない参考書は、問題番号だけでは特定できません。自習では「参考書名・版・ページ・問題番号」と、質問に必要な問題文・コード・条件・選択肢を伝えます。番号だけでAIに内容を推測させないことが大切です。

利用できる資料とAIの機能に合わせ、次の三つの方法から選びます。アップロードや共有が認められている資料を使いましょう。

1

**文章をコピーして貼る**  
WebページやPDFで文字を選択できる場合

2

**スクリーンショットを添付する**  
図・表・擬似言語を含む問題の場合

3

**必要な部分を手で入力する**  
コピーも画像添付もできない場合

### 方法1 問題文をコピーして貼り付ける

問題文、擬似言語、選択肢をまとめてコピーし、質問の前に貼り付けます。長い問題では、次の見出しを付けるとAIが内容を区別しやすくなります。

| AIへ貼り付ける形   |                             |
|-------------|-----------------------------|
| 問題文         | ここに問題文を貼る                   |
| 擬似言語        | ここに擬似言語を貼る                  |
| 選択肢         | ここに選択肢を貼る                   |
| 自分が分からないところ | 例: 3行目で、totalにどの値が入るのか分からない |

### 方法2 スクリーンショットを添付する

文字を選択できない場合は、問題全体が読めるスクリーンショットを添付します。問題文、擬似言語、選択肢が途中で切れないようにしてください。



### 画像を添付したら、最初に読み取り確認をする

AIは、画像内の添字、記号、数字を読み間違えることがあります。すぐに解かせず、「画像から読み取った問題文と擬似言語をそのまま書き出してください」と頼み、元画像と見比べます。

## 方法3 必要な部分を手で入力する

画像を添付できない場合は、問題の条件と擬似言語を入力します。すべてを打ち直すのが大変なら、分からない行と、その行で使う変数の初期値だけでも構いません。

### AIへ送る順番

問題を渡す



読み取りを確認する



分からない点を書く



ヒントを頼む

## 0.5 研修中・自習に共通する質問の工夫

研修中は、次の例文の先頭に「本教材の演習2-Bについて」のように問題番号を付けます。研修後の別の参考書では、0.4の方法で対象の問題を伝え、読み取りを確認してから使います。自分で書いた解答やトレース表は、共有PDFには入っていないため、自分で入力するか画像で添付します。

AI

### AIへの質問例 | ヒントが欲しいとき

あなたはプログラミング未経験者の学習支援者です。指定した問題について、正解や完成した擬似言語はまだ示さないでください。最初に確認すべきことを一つだけ、私に質問してください。私が答えたら次の質問へ進んでください。

AI

### AIへの質問例 | 自分の考えを点検するとき

これから私の考え方を説明します。正解を先に示さず、論理が最初に成立しなくなる箇所だけを指摘してください。その理由は一文で説明してください。

指定した問題と同じ考え方で解ける類題を一問作ってください。AIが作成した問題であることを明記し、条件に矛盾がないことを確認してください。解答と解説は私が回答するまで表示しないでください。

## 0.6 AIの回答を確認する

AIから回答を得たら、次の順番で確認します。

1. 問題文に書かれていない条件を勝手に追加していないか
2. 配列の先頭の添字を取り違えていないか
3. 繰返しの開始値・終了値を取り違えていないか
4. 代入前の値と代入後の値を混同していないか
5. 自分でトレースして同じ結果になるか

### 個人情報・会社情報を入力しない

氏名、メールアドレス、顧客情報、公開されていない会社資料、パスワードなどはAIへ入力しません。この教材の練習問題のように、公開しても問題のない内容だけを使います。

# 第1章 アルゴリズムと擬似言語

## 1.1 アルゴリズムとは

アルゴリズムとは、目的を達成するための**処理手順**です。

身近なたとえ

アルゴリズムは「料理のレシピ」と同じ

カレーを作るとき、材料だけを渡されても完成しません。「切る → 炒める → 煮る」のように、作業の順番が必要です。コンピュータにも、同じように具体的な手順を一つずつ伝えます。



例えば、「三つの数の中から最大の数を見つける」という目的に対して、次の手順を考えられます。

1. 一つ目の数を、現在の最大値として覚える
2. 二つ目の数が現在の最大値より大きければ、最大値を更新する
3. 三つ目の数も同じように比較する
4. 最後に残った最大値を表示する

人間が何となく行っている判断を、コンピュータが実行できる順序に分解したものがアルゴリズムです。

例: 3、10、7の中から最大値を探す

3

最初の暫定1位

10

3より大きいので、暫定1位を10へ交代

7

10より小さいので、そのまま

最後の暫定1位「10」が答え

## 1.2 良いアルゴリズム

アルゴリズムには、少なくとも次の性質が必要です。

- 手順が明確である
- 同じ入力に対して、決められた結果になる
- いつか処理が終了する
- 実行できない曖昧な指示がない

「いい感じに並べる」「適切な回数だけ繰り返す」という表現は、人間には伝わってもコンピュータには伝わりません。

人には通じるかもしれない

「数字をいい感じに並べて」

何を基準に？ 大きい順？ 小さい順？

コンピュータにも伝わる

「左から右へ、小さい順に並べる」

基準と終了地点が明確

## 1.3 擬似言語とは

擬似言語は、アルゴリズムをプログラムに近い形で表現するための記述方法です。特定のプログラミング言語に依存せず、処理の考え方を表します。



本書では、IPAが定める基本情報技術者試験用の擬似言語の記述形式に合わせます。

## はじめて出てくる記号と言葉の読み方

次の例には、これから何度も使う基本の書き方が含まれています。分からない英単語や記号があっても、最初は一つずつ意味を対応させれば大丈夫です。

| 書き方           | 読み方・役割                                                          |
|---------------|-----------------------------------------------------------------|
| 整数型: price    | price という名前の「整数を入れる箱」を用意する。整数型は、小数点のない数( 0、150、-3 など)を入れるという意味。 |
| price ← 150   | 右側の 150 を、左側の箱 price に入れる。← は「代入(だいにゅう)」といい、等しいという意味ではない。       |
| price × count | price の値と count の値を掛け算する。× は掛け算、+ は足し算、÷ は割り算。                  |
| 表示する(total)   | total に入っている値を画面に見せる。丸括弧の中は、表示する対象。                             |

ここでは、price、count、total を変数と呼びます。変数とは、値を覚えておくための「名前付きの箱」です。英単語そのものを暗記する必要はありません。何を入れている箱かを考えて読んでください。

```
整数型: price
整数型: count
整数型: total

price ← 150
count ← 3
total ← price × count
表示する(total)
```

上から一行ずつ実行すると、total には450が入り、450が表示されます。

※ 本書の練習問題で使う共通仕様

関数は、値を受け取って計算し、結果の値を返す部品です。手続は、表示などの作業を行う部品です。本書では 入力する() を「入力された値を返す関数」、表示する(値) を「受け取った値を画面に表示する手続」として扱います。これらの名前は試験共通の命令ではありません。実際の試験では、問題文に書かれた関数・手続の名前と仕様に従います。

## ※「○」は手続・関数の宣言

```
○整数型: add(整数型: a, 整数型: b)  
    return a + b
```

宣言は、「ここからこの部品を定義します」と示す書き方です。試験では、手続や関数の宣言を先頭の ○ で表します。この例は、整数 *a* と *b* を受け取り、その合計を返す関数です。*return* は「この値を結果として返す」という意味です。

## 問題文で定義された仕様を最優先する

本書のコード表記は試験形式に合わせています。ただし、配列の要素番号が始まる位置、関数が受け取る引数、返す値などは問題ごとに定義されます。その部分は必ず問題文の指示を優先します。

# 1.4 処理を作る三つの基本構造

## 解説のフローチャートの読み方

フローチャートは、処理の進み方を図で表したものです。まず「開始」から矢印を一つずつたどります。

| 図の形     | 意味       | 読み方                      |
|---------|----------|--------------------------|
| 丸い開始・終了 | 処理の入口と出口 | 開始から読み、終了で止める            |
| 四角い箱    | 実行する処理   | 箱の中を上から順に行う              |
| ひし形     | 条件判定     | 「はい(真)」か「いいえ(偽)」の片方だけへ進む |
| 矢印      | 次に進む先    | 上へ戻る矢印なら、戻った条件をもう一度確かめる  |

繰返しもひし形で「まだ続けるか」を確認します。forの図は、最初の値を設定→条件を確認→中の処理→値を1増やす→条件へ戻る、に分けて描いています。forなどの読み方は、この後の説明と第5章でも練習します。

複雑なアルゴリズムも、基本的には次の三つを組み合わせで作ります。

1

## 順次

上から順番に行く

例: 服を着てから靴を履く

2

## 選択

条件で行動を変える

例: 雨なら傘を持つ

3

## 繰返し

同じことを何度か行う

例: 全員分の出席を取る

## 順次

上から下へ、決められた順番で処理します。

```

a ← 5
b ← 10
c ← a + b

```

## 選択

条件によって、実行する処理を分けます。

条件とは、「はい」か「いいえ」で答えられる質問です。例えば、`score ≥ 60` は「`score` は60以上か」という質問です。`if` から `endif` までが一组の選択処理です。

| 書き方                  | 意味                                               |
|----------------------|--------------------------------------------------|
| <code>if (条件)</code> | 条件が成立したときに、その直後の処理を行う。 <code>if</code> は「もし～なら」。 |
| <code>else</code>    | <code>if</code> の条件が成立しなかったときに行う処理へ切り替える。        |
| <code>endif</code>   | 選択処理の終わりを示す。                                     |

```

if (score ≥ 60)
  表示する("合格")
else
  表示する("不合格")
endif

```

## 繰返し

条件や回数に従って、同じ処理を繰り返します。

for から endfor までは一組の繰返し処理です。i は、今何回目かを数えるための変数で、ここでは1、2、3、4、5と変化します。

| 書き方                         | 意味                                         |
|-----------------------------|--------------------------------------------|
| for (i を 1 から 5 まで 1 ずつ増やす) | i を1から始め、1ずつ増やしながら5まで同じ処理を繰り返す。5のときも処理を行う。 |
| endfor                      | 繰返し処理の終わりを示す。                              |

```
for (i を 1 から 5 まで 1 ずつ増やす)
  表示する(i)
endfor
```

#### 理解チェック

「合計が100以上なら割引し、商品ごとに同じ計算を繰り返す」という処理には、三つの基本構造のうち何が含まれますか。

## 演習1-A 朝の手順

難易度 ★☆☆

「靴を履く→靴下を履く→外へ出る」を、やり直しが起きない順序に直してください。三つの基本構造のどれですか。

自分で書く：答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習1-Aについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。



## 演習1-B 雨の日の持ち物

---

難易度 ★☆☆

雨なら傘を持ち、雨でなければ帽子を持って出発します。雨の日に通る処理と、三つの基本構造の名前を答えてください。

自分で書く： 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習1-Bについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習1-C 作業が終わる条件

---

難易度 ★☆☆

未確認の申込書がある間、一枚取り出して確認済みにします。未確認が3枚なら確認は何回ですか。最初から0枚ならどうなりますか。

自分で書く： 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習1-Cについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習1-D 判定後の合流

---

難易度 ★☆☆

利用証が有効なら入館を許可し、無効なら受付へ案内します。どちらの場合も対応記録を残します。記録を「有効な場合だけ」に置くと何が困りますか。

自分で書く： 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習1-Dについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習1-E 出荷の確認手順

---

難易度 ★★★

【実践】注文を一件処理します。在庫があれば商品を確認して出荷連絡、なければ入荷待ち連絡をします。最後に必ず対応履歴を保存します。処理手順を日本語で書き、在庫なしの場合をたどってください。

自分で書く：答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習1-Eについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習1-F 申込書をまとめて点検する

---

難易度 ★★★

【実践】未処理の申込書がある間、一枚ずつ点検します。不備があれば修正依頼、なければ受付完了を連絡し、必ず処理済みにします。この手順を書いてください。「処理済みにする」を忘れた場合の問題も説明してください。

自分で書く：答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習1-Fについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

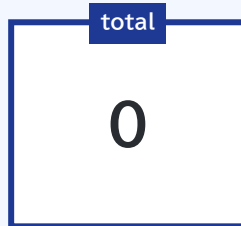
## 第2章 変数・データ型・代入

### 2.1 変数は値を覚える場所

プログラムでは、計算途中の値を覚えておく必要があります。その保管場所が**変数**です。

変数には名前を付けます。例えば、合計を保存する変数なら `total`、件数なら `count` のように、役割が分かる名前を使います。

変数は「名前付きの箱」



箱の名前: `total`

箱の中身: `0`

箱に入れられるもの: 整数

```
整数型: total  
total ← 0
```

この例では、整数を保存する変数 `total` を用意し、最初の値として0を入れています。

### 2.2 データ型

データ型は、変数にどのような値を保存するかを表します。「型」は値の種類という意味です。

データ型は、箱に貼る「中に何を入れてよいか」というラベルです。整数用の箱に文章を入れないように、保存する値の種類をあらかじめ決めます。

| データ型 | 保存する値の例      | 使用例      |
|------|--------------|----------|
| 整数型  | 10、-3、0      | 件数、添字、合計 |
| 実数型  | 3.14、18.5    | 平均、身長、割合 |
| 文字型  | 'A'、'？'      | 一文字の記号   |
| 文字列型 | "合格"、"Tokyo" | 名前、メッセージ |
| 論理型  | true、false   | 条件が成立するか |

実数型は、小数点を含めて扱える数を入れる型です。例えば身長 1.68 m、平均 72.5 点、消費税率 0.1 のような値に使います。論理型の true は「条件が成立する」、false は「条件が成立しない」という意味で、後の if や while の判定に使います。

### 数字と文字列は別物

整数の 10 は計算に使えますが、文字列の "10" は文字として扱われます。問題文で指定されたデータ型を確認しましょう。

## 2.3 代入は右から左へ読む

代入とは、計算結果や値を変数へ入れる操作です。

```
total ← total + 5
```

右側を先に計算

$3 + 5 = 8$

→

左側の箱へ入れる

$total \leftarrow 8$

これは、次の順番で実行します。

1. 現在の total の値を取り出す
2. その値に5を足す
3. 計算結果を total へ入れ直す

数学の等式ではありません。「左辺と右辺が等しい」とは読みません。

## 例

実行前の `total` が3の場合を考えます。

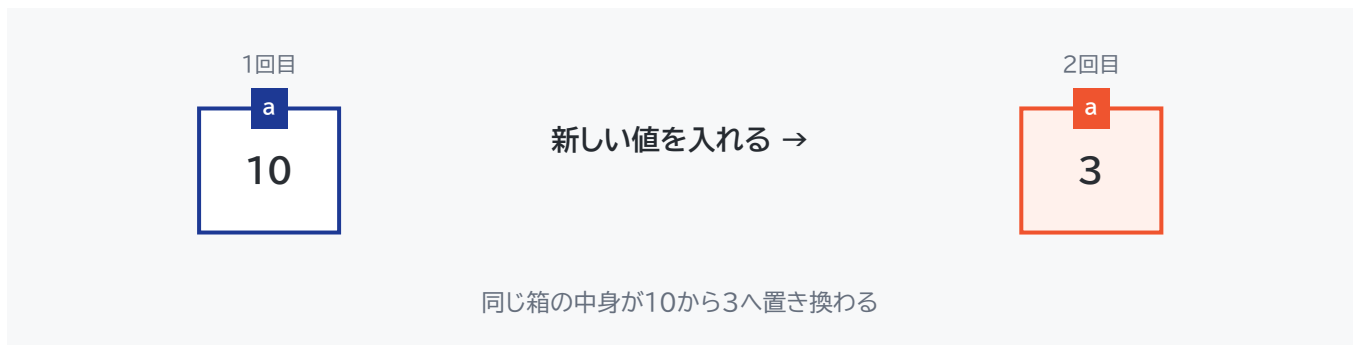
| 実行する文                          | 右辺の計算              | 実行後の <code>total</code> |
|--------------------------------|--------------------|-------------------------|
| <code>total ← total + 5</code> | <code>3 + 5</code> | 8                       |

## 2.4 値は上書きされる

変数へ新しい値を代入すると、以前の値は失われます。

```
a ← 10  
a ← 3
```

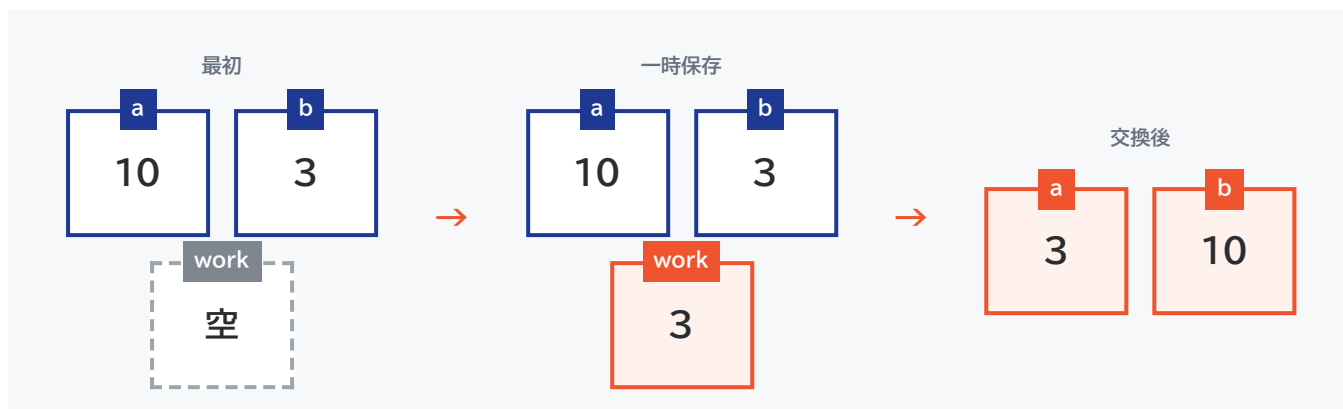
実行後の `a` は3です。最初に入れた10は残りません。



## 2.5 二つの変数の値を交換する

`a` に10、`b` に3が入っているとします。二つの値を交換するには、一時的に値を保存する変数が必要です。

二つのコップの飲み物を交換するとき、空のコップが一つ必要なのと同じです。`work` は、値を一時的に避難させる空のコップです。



整数型: work

```
work ← b
b ← a
a ← work
```

| 実行後      | a  | b  | work |
|----------|----|----|------|
| 初期状態     | 10 | 3  | 未定義  |
| work ← b | 10 | 3  | 3    |
| b ← a    | 10 | 10 | 3    |
| a ← work | 3  | 10 | 3    |

### よくある間違い

```
a ← b
b ← a
```

一行目を実行した時点で、`a` に入っていた10が失われます。二行目では、どちらも3になってしまいます。

## 演習2-A 代入を追う

難易度 ★☆☆

次の処理が終わったとき、`total` の値はいくつですか。

整数型: `total`

```
total ← 2
```

```
total ← total + 3
```

```
total ← total × 2
```

| 実行した文                          | 実行後の <code>total</code> |
|--------------------------------|-------------------------|
| <code>total ← 2</code>         |                         |
| <code>total ← total + 3</code> |                         |
| <code>total ← total × 2</code> |                         |

AI

### AIへの質問例 | 値の変化を追えないとき

最終的な答えはまだ教えないでください。各行の右辺を先に計算するために、私へ一問ずつ質問してください。私の回答が間違っている場合は、その行だけを指摘してください。

## 演習2-B 値を交換する

難易度 ★☆☆

`x` に7、`y` に12が入っています。実行後に `x` が12、`y` が7となるように、空欄を埋めてください。

整数型: `work`

```
work ← [ ① ]
```

```
x ← [ ② ]
```

```
y ← [ ③ ]
```

### 説明してみよう

なぜ一時変数 `work` が必要なのかを、プログラミングを知らない人へ説明してください。

## 演習2-C 同じ名前を何度も更新する

難易度 ★★☆☆

最後のaとbを答え、各行の実行後の値を表にしてください。

この問題の変数は整数型です。

```
a ← 4
b ← a
a ← a + 6
b ← b + a
```

自分で書く: 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例: 本教材の演習2-Cについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習2-D 小数を含む計算

難易度 ★★☆☆

price、rate、discountは実数型です。割引額discountと割引後のpriceを教えてください。rate=0.2は20%を表します。

実数型: price, rate, discount

```
price ← 1250
rate ← 0.2
discount ← price × rate
price ← price - discount
```

自分で書く: 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例: 本教材の演習2-Dについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。



## 演習2-E 口座間の資金移動

難易度 ★★★

【実践】手数料なしで口座aから口座bへamount円移します。aの残高は移動額以上とします。処理後の両残高と合計を求め、合計が変わらない理由を説明してください。

この問題の変数は整数型です。

```
a ← 5000
b ← 2000
amount ← 1200
a ← a - amount
b ← b + amount
total ← a + b
```

自分で書く: 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例: 本教材の演習2-Eについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習2-F 担当番号を一つずらす

難易度 ★★★

【実践】窓口a、b、cの担当番号を、aには元のb、bには元のc、cには元のaが入るよう更新します。最後の三つの値を教えてください。workを使わず最後に $c \leftarrow a$ にすると何が起きますか。

この問題の変数は整数型です。

```
a ← 10  
b ← 20  
c ← 30  
work ← a  
a ← b  
b ← c  
c ← work
```

自分で書く：答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習2-Fについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 第3章 順次処理

### 3.1 上から順番に実行する

順次処理は、文を上から下へ一度ずつ実行する、最も基本的な構造です。

① リンゴの単価を150円として覚える



② みかんの単価を80円として覚える



③ それぞれの代金を計算して足す



④ 合計金額を表示する

整数型: apple

整数型: orange

整数型: total

apple ← 150

orange ← 80

total ← apple × 3 + orange × 4

表示する(total)

この処理は、150円のリンゴを3個、80円のみかんを4個買ったときの合計金額を求めています。

### 3.2 入力・処理・出力で整理する

問題文を読んだら、最初に次の三つへ分けます。



| 分類 | 意味         | 例        |
|----|------------|----------|
| 入力 | 最初に与えられる値  | 単価、個数    |
| 処理 | 入力を使って行う計算 | 単価×個数、合計 |
| 出力 | 最後に求めるもの   | 合計金額     |

AI

AIへの質問例 | 問題文を整理するとき

完成した擬似言語は作らないでください。次の問題文から、入力・処理・出力を分けるための質問を、一問ずつ私にしてください。私が答えた後、不足があれば指摘してください。

演習3-A 代金を計算する

難易度 ★☆☆

一つ150円のリンゴを3個、一つ80円のみかんを4個買います。合計金額を計算して表示するように、空欄を埋めてください。

```
整数型: apple
整数型: orange
整数型: total

apple ← [ ① ]
orange ← [ ② ]
total ← [ ③ ]
表示する(total)
```

## 自分の説明

自分の言葉で書く

変数 `apple` の役割

-----

変数 `orange` の役割

-----

変数 `total` の役割

-----

ペアまたはAIへ説明しよう

自分が作った式を、どの部分が「リンゴの代金」「みかんの代金」「全体の合計」なのかに分けて説明してください。聞き手は、答えを教えるのではなく、変数の役割について質問します。

## 演習3-B 消費税込みの金額

難易度 ★★☆☆

商品の税抜価格 `price` と税率 `rate` が与えられています。税込価格を求めて表示する処理を作成してください。小数点以下の扱いは考えず、実数型で計算します。

実数型: `price`

実数型: `rate`

実数型: `taxIncluded`

`price` ← 1200

`rate` ← 0.1

// ここに処理を書く

AI

AIへの質問例 | 式を点検するとき

私が作った税込価格の計算式を確認してください。正しい式を先に示さず、「税額を求めている式」か「税込価格を求めている式」という観点だけで指摘してください。

## 演習3-C 入力から出力まで

難易度 ★☆☆

priceは入力された単価、quantityは入力された個数とします。単価200円、個数3個のとき何が表示されますか。入力・処理・出力に分けて説明してください。

この問題の変数は整数型です。

```
price ← 200
quantity ← 3
total ← price × quantity
表示する(total)
```

自分で書く：答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習3-Cについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習3-D 計算する順番

難易度 ★☆☆

最後のpriceを求めてください。「割引後に送料を足す」と「送料を足してから割り引く」が同じにならない理由も説明してください。金額は実数型で、小数点以下の丸めは行いません。

実数型: price

```
price ← 2000
price ← price × 0.9
price ← price + 300
```

自分で書く：答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習3-Dについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習3-E 作業時間の見積り

難易度 ★★★

【実践】製品を18個作り、一個に7分、準備に12分かかります。総時間minutesと時間単位のhoursを求めてください。hoursは実数型で、小数点以下を切り捨てません。準備を個数倍してはいけない理由も説明してください。

整数型: quantity, perItem, setup, minutes 実数型: hours

```
quantity ← 18
perItem ← 7
setup ← 12
minutes ← quantity × perItem + setup
hours ← minutes ÷ 60
```

自分で書く: 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例: 本教材の演習3-Eについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習3-F 倉庫の一日の記録

難易度 ★★★

【実践】朝の在庫40個に15個入荷し、23個出荷しました。出荷可能な在庫があるものとします。一日の出荷後在庫と、翌日の目標50個に対する不足数を求めてください。

この問題の変数は整数型です。

```
stock ← 40
received ← 15
shipped ← 23
stock ← stock + received
stock ← stock - shipped
shortage ← 50 - stock
```

自分で書く: 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例: 本教材の演習3-Fについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

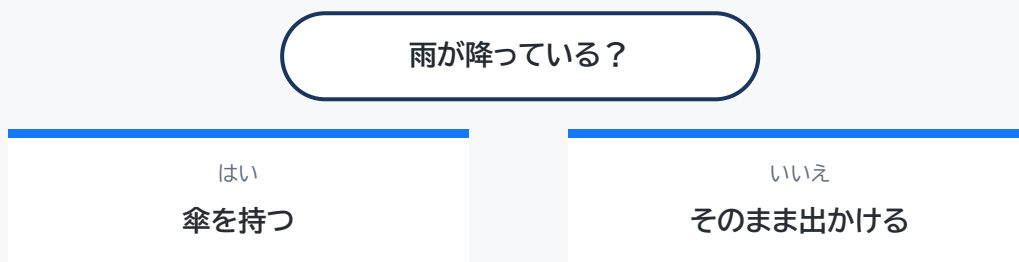


## 第4章 選択処理

### 4.1 条件によって処理を分ける

選択処理は、条件が成立するかどうかによって、実行する処理を変えます。

選択は「雨が降っている？」と考えるのと同じ



```
if (score ≥ 60)
    表示する("合格")
else
    表示する("不合格")
endif
```

条件 `score ≥ 60` が成立すれば「合格」、成立しなければ「不合格」を表示します。`if`、`else`、`endif` の役割は第1章で確認しましたが、コードを上から読むと、(1) 条件を判定する、(2) 成立なら `if` の直後を実行する、(3) 成立しないなら `else` の直後を実行する、(4) `endif` で選択処理を終える、という順番です。

## 4.2 比較演算子

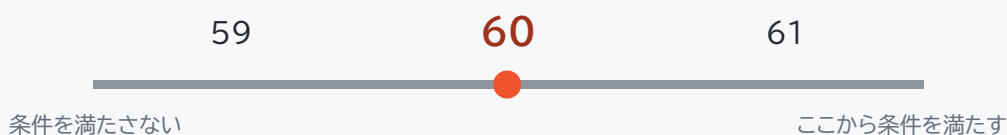
| 演算子               | 意味    | 例                      |
|-------------------|-------|------------------------|
| <code>=</code>    | 等しい   | <code>x = 10</code>    |
| <code>≠</code>    | 等しくない | <code>x ≠ 10</code>    |
| <code>&gt;</code> | より大きい | <code>x &gt; 10</code> |
| <code>&lt;</code> | より小さい | <code>x &lt; 10</code> |
| <code>≧</code>    | 以上    | <code>x ≧ 10</code>    |
| <code>≦</code>    | 以下    | <code>x ≦ 10</code>    |

### ※ 試験で使う関係演算子

キーボード入力によく使う `>=` や `<=` ではなく、試験問題では `≧` や `≦` を使います。代入は `<-`、等しいかの比較は `=` です。

「18より大きい」と「18以上」は異なります。境界となる値を確認しましょう。

### 「60点以上」の境界を見る



## 4.3 複数の条件

二つ以上の条件を組み合わせるときは、論理演算を使います。

### and

学生証も必要  
予約票も必要

両方そろって通れる

### or

学生証または  
運転免許証

どちらか一方で通れる

### not

「雨ではない」

条件を反対にする

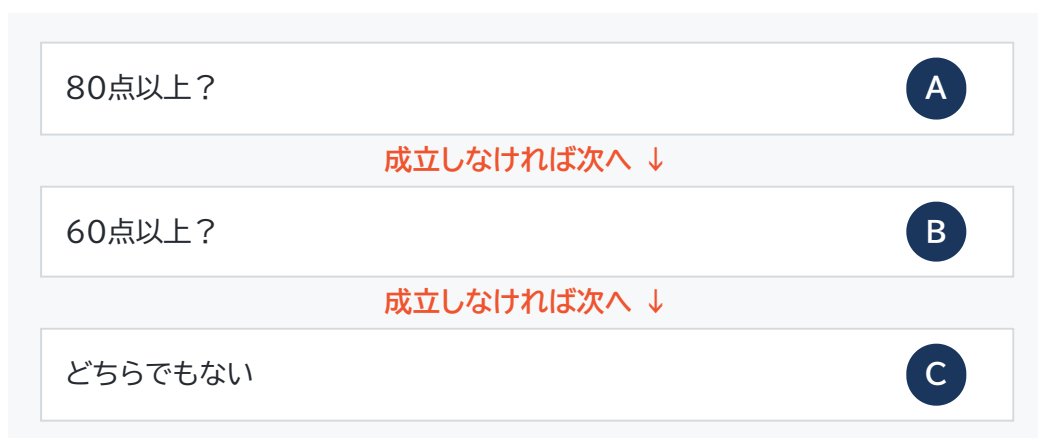
| 論理演算 | 成立する条件     |
|------|------------|
| and  | 両方の条件が成立   |
| or   | 少なくとも一方が成立 |
| not  | 条件が成立しない   |

```
if (age ≥ 18 and age < 65)
    表示する("対象")
endif
```

## 4.4 三つ以上に分ける

```
if (score ≥ 80)
    表示する("A")
elseif (score ≥ 60)
    表示する("B")
else
    表示する("C")
endif
```

上から条件を確認し、最初に成立した処理だけを実行します。そのため、条件を書く順番が重要です。



### 順番を逆にするとどうなるか

```
if (score ≥ 60)
    表示する("B")
elseif (score ≥ 80)
    表示する("A")
endif
```

score が90でも、最初の  $\text{score} \geq 60$  が成立するため「B」と表示されます。

## 演習4-A 偶数・奇数を判定する

難易度 ★☆☆

整数  $x$  が偶数なら「偶数」、そうでなければ「奇数」と表示します。空欄を埋めてください。 $x \bmod 2$  は、 $x$  を2で割った余りです。

```
if ([ ① ])
    表示する("偶数")
else
    表示する("奇数")
endif
```

## 演習4-B BMIを判定する

難易度 ★☆☆

身長  $\text{height}$  (m)と体重  $\text{weight}$  (kg)からBMIを求め、次の基準で結果を表示します。

### BMIの計算式

$$\text{BMI} = \text{体重} \div (\text{身長} \times \text{身長})$$

18.5未満  
低体重

18.5以上25未満  
標準体重

25以上  
肥満

```
実数型: height
実数型: weight
実数型: bmi
文字列型: result

height ← 入力する()
weight ← 入力する()
bmi ← weight ÷ (height × height)

if ([ ① ])
  result ← "低体重"
elseif ([ ② ])
  result ← "標準体重"
else
  result ← "肥満"
endif

表示する(result)
```

## 境界値を確認する

| bmi   | 期待する結果 |
|-------|--------|
| 18.49 |        |
| 18.50 |        |
| 24.99 |        |
| 25.00 |        |



### AIへの質問例 | 条件漏れを点検するとき

完成した条件式は表示しないでください。私が書いたBMI判定について、18.5と25.0の境界値を使って判定漏れや重複がないか確認し、問題がある条件だけを指摘してください。

### 説明してみよう

二つ目の条件を `bmi < 25` だけで書ける理由を説明してください。最初の `if` が成立しなかった時点で、どの条件が分かっているのでしょうか。

## 演習4-C 合格の境目

難易度 ★☆☆

scoreに59、60をそれぞれ入れたときの表示を教えてください。≥は「以上」を表します。

整数型: score 文字列型: result

```
if (score ≥ 60)
    result ← "合格"
else
    result ← "再挑戦"
endif
表示する(result)
```

自分で書く: 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例: 本教材の演習4-Cについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習4-D 二つの条件を満たす

難易度 ★★☆☆

ageは年齢、hasTicketは券ありなら1、なしなら0です。入場には12歳以上かつ券ありが必要です。(age, hasTicket)が(12,1)、(11,1)、(20,0)の結果を教えてください。

整数型: age, hasTicket 文字列型: result

```
if (age ≥ 12 and hasTicket = 1)
    result ← "入場可"
else
    result ← "入場不可"
endif
```

自分で書く: 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例: 本教材の演習4-Dについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習4-E 利用料金を判定する

難易度 ★★★

【実践】年齢ageは0以上の整数です。6歳未満は無料、6歳以上18歳未満は400円、18歳以上は900円です。age=5、6、17、18のfeeを教えてください。二つ目の条件をage≤18に変えると仕様に合わない入力はどうですか。

この問題の変数は整数型です。

```
if (age < 6)
    fee ← 0
else
    if (age < 18)
        fee ← 400
    else
        fee ← 900
    endif
endif
```

自分で書く： 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習4-Eについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。



## 演習4-F 配送サービスを選ぶ

難易度 ★★★

【実践】冷蔵が必要(cold=1)なら金額によらず冷蔵便600円。不要(cold=0)なら購入額amountが5000円以上で送料無料、未満で通常便300円です。(cold, amount)=(1,6000)、(0,5000)、(0,4999)の送料を求め、金額から判定するときの注意点を説明してください。

この問題の変数は整数型です。

```
if (cold = 1)
  fee ← 600
else
  if (amount ≥ 5000)
    fee ← 0
  else
    fee ← 300
  endif
endif
```

自分で書く：答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習4-Fについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 第5章 繰り返し処理

### 5.1 同じ処理を繰り返す

繰り返し処理は、決められた回数または条件が成立している間、同じ処理を実行します。

#### 5人の出席を確認する



「名前を呼ぶ → 返事を記録する」を5回繰り返す

```
for (i を 1 から 5 まで 1 ずつ増やす)
  表示する(i)
endfor
```

この処理は、1、2、3、4、5を順番に表示します。

### 5.2 回数が決まっている繰り返し

`for` は、繰り返す回数が分かっている場合に向いています。英単語の `for` は「～の間」という感覚で、ここでは「`i` を指定した範囲で変化させる間、同じ処理を行う」と読めば大丈夫です。



整数型: `i`

```
for (i を 1 から 10 まで 1 ずつ増やす)
  表示する(i)
endfor
```

※「1から10まで」は10を含む

for (i を 1 から 10 まで 1 ずつ増やす) では、i は1、2、…、10と変化します。開始値・終了値・増減の方向を、毎回確認します。

## for の一行を分解して読む

for (i を 1 から 10 まで 1 ずつ増やす) は、次の四つを一度に指定しています。

1. 数える変数: i を使う
2. 開始値: i を1から始める
3. 終了値: i が10のときまで続ける
4. 変化量: 1回ごとに i を1ずつ増やす

したがって、この例の処理本体は10回実行されます。i の値を横に書き出すと、繰返しを追いやすくなります。

## 5.3 条件で続ける繰返し

while は、条件が成立している間、処理を繰り返します。while は「～である間」という意味です。while (i ≤ 5) なら、「i が5以下である間、endwhile までの処理を繰り返す」と読みます。endwhile は、この繰返し処理の終わりを示します。

身近なたとえ: 行列に人がいる間は受付を続ける

for: 最初から「10人」と分かっているとき

while: あと何人来るか分からず、「待っている人がいる間」続けるとき

整数型: i

```
i ← 1
while (i ≤ 5)
  表示する(i)
  i ← i + 1
endwhile
```

## while を読む順番

1. 繰返しの前に初期値を確認する
2. 条件式を確認する
3. 条件が成立したら処理を実行する
4. 変数の更新を確認する
5. 条件式へ戻る

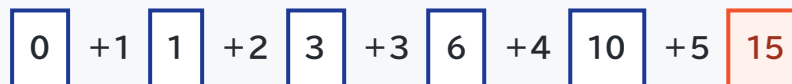
### 無限ループに注意

$i \leftarrow i + 1$  がなければ、 $i$  は1のままです。条件  $i \leq 5$  が常に成立し、処理が終わりません。繰返しでは「いつ条件が成立しなくなるか」を確認します。

## 5.4 合計を求める

1から5までの合計を求めます。

**total** は、数字を貯める貯金箱



整数型:  $i$

整数型: total

```
total ← 0
```

```
for ( $i$  を 1 から 5 まで 1 ずつ増やす)
```

```
  total ← total +  $i$ 
```

```
endfor
```

表示する(total)

| i | 実行前の total | total + i | 実行後の total |
|---|------------|-----------|------------|
| 1 | 0          | 1         | 1          |
| 2 | 1          | 3         | 3          |
| 3 | 3          | 6         | 6          |
| 4 | 6          | 10        | 10         |
| 5 | 10         | 15        | 15         |

### 累積する変数

total のように、繰返しのたびに値を加えていく変数を累積用の変数として使います。合計の初期値は通常0です。

## 5.5 件数を数える

1から10までの整数のうち、偶数の件数を数えます。

count は、条件に合ったときだけ押すカウンター



色の付いた偶数だけで、カウンターを1増やす

整数型: i

整数型: count

```
count ← 0
```

```
for (i を 1 から 10 まで 1 ずつ増やす)
```

```
  if (i mod 2 = 0)
```

```
    count ← count + 1
```

```
  endif
```

```
endfor
```

表示する(count)

`count` は、条件が成立した場合だけ1増えます。

## 演習5-A 合計と平均

難易度 ★★☆☆

1から10までの整数を順番に表示し、最後に合計と平均を表示します。空欄を埋めてください。

```
整数型: i
整数型: total
実数型: average

total ← [ ① ]

for (i を [ ② ] から [ ③ ] まで 1 ずつ増やす)
  表示する(i)
  total ← [ ④ ]
endfor

average ← [ ⑤ ]
表示する(total)
表示する(average)
```

AI

### AIへの質問例 | 繰返しを追うとき

空欄の答えは教えないでください。`i`、実行前の `total`、実行後の `total` を記録する空のトレース表を作ってください。最初の二回は私が埋めるので、間違っているセルだけを指摘してください。

## 演習5-B 処理回数を数える

難易度 ★★☆☆

次の処理で、処理①と処理②はそれぞれ何回実行されますか。

整数型:  $i$

$i \leftarrow 1$

while ( $i \leq 5$ )

  // 処理①

  if ( $i \bmod 2 = 1$ )

    // 処理②

  endif

$i \leftarrow i + 1$

endwhile

| $i$ | $i \leq 5$ | 処理① | $i \bmod 2 = 1$ | 処理② |
|-----|------------|-----|-----------------|-----|
| 1   |            |     |                 |     |
| 2   |            |     |                 |     |
| 3   |            |     |                 |     |
| 4   |            |     |                 |     |
| 5   |            |     |                 |     |
| 6   |            |     |                 |     |

AI

### AIへの質問例 | 自分の表を点検するとき

実行回数の正解はまだ教えないでください。私が作成した表を確認し、条件判定と処理実行を混同している最初の行だけを指摘してください。

## 演習5-C 三回足す

難易度 ★☆☆

最後のtotalを求め、繰返し部分が何回実行されるか教えてください。

この問題の変数は整数型です。

```
total ← 0
for (i を 1 から 3 まで 1 ずつ増やす)
  total ← total + 2
endfor
```

自分で書く： 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習5-Cについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習5-D 最初の判定が偽なら

難易度 ★☆☆

表示されるiはいくつですか。繰返し内部は何回実行されますか。

この問題の変数は整数型です。

```
i ← 5
while (i < 5)
  i ← i + 1
endwhile
表示する(i)
```

自分で書く： 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習5-Dについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。



## 演習5-E 目標金額まで積み立てる

難易度 ★★★

【実践】最初の貯金は1000円、毎月700円を加えます。3000円以上になるまで何か月必要ですか。最後の貯金額と、条件判定の回数も答えてください。

この問題の変数は整数型です。

```
balance ← 1000
months ← 0
while (balance < 3000)
  balance ← balance + 700
  months ← months + 1
endwhile
```

自分で書く：答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習5-Eについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習5-F 一括処理の所要時間

難易度 ★★★

【実践】1番から6番までの伝票を順に処理します。各伝票に2分、3番と6番には追加点検で各5分かかります。合計時間を求めてください。 $i \bmod 3 = 0$ は、番号が3の倍数かを判定します。

この問題の変数は整数型です。

```
minutes ← 0
for (i を 1 から 6 まで 1 ずつ増やす)
  minutes ← minutes + 2
  if (i mod 3 = 0)
    minutes ← minutes + 5
  endif
endfor
```

自分で書く：答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習5-Fについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

# 第6章 配列

## 6.1 配列は同じ型の値をまとめる

配列は、同じデータ型の複数の値を、一つの名前で管理する仕組みです。例えば、10人分の点数を一つずつ別の変数に入れる代わりに、`A` という一つの配列にまとめます。

配列は「番号付きのロッカー」

|      |      |      |
|------|------|------|
| A[1] | A[2] | A[3] |
| 5    | 10   | 3    |

配列名は建物の名前、添字はロッカー番号、値は中に入っている荷物です。

整数型の配列: `A ← {5, 10, 3}`

この例では、配列の要素番号を1から始めます。

| 要素 | A[1] | A[2] | A[3] |
|----|------|------|------|
| 値  | 5    | 10   | 3    |

`A` は配列全体の名前、`A[2]` は二番目の要素を表します。角括弧の中の番号を\*\*添字(そえじ)\*\*といい、どの箱を使うかを指定する番号です。`A[2]` は「Aの2番の箱」と読んでください。

### ※ 配列の要素番号を確認する

試験問題では、「配列の要素番号は1から始まる」のように開始位置が示されます。`A[1]` を先頭と決めつけず、問題文の指定を確認します。

## 6.2 配列の要素を順番に表示する

整数型:  $i$

整数型の配列:  $A \leftarrow \{3, 8, 2, 5\}$

for ( $i$  を 1 から 4 まで 1 ずつ増やす)

表示する( $A[i]$ )

endfor

$i$  が 1、2、3、4 と変化するため、 $A[1]$ 、 $A[2]$ 、 $A[3]$ 、 $A[4]$  を順番に参照します。

| $A[1]$ | $A[2]$ | $A[3]$ | $A[4]$ |
|--------|--------|--------|--------|
| 3      | 8      | 2      | 5      |

↑  $i = 2$  なら、2 番のロッカー  $A[2]$  を見る

## 6.3 配列の合計

整数型:  $i$

整数型:  $total$

整数型の配列:  $A \leftarrow \{3, 8, 2, 5\}$

$total \leftarrow 0$

for ( $i$  を 1 から 4 まで 1 ずつ増やす)

$total \leftarrow total + A[i]$

endfor

表示する( $total$ )

| i | A[i] | 実行前の total | 実行後の total |
|---|------|------------|------------|
| 1 | 3    | 0          | 3          |
| 2 | 8    | 3          | 11         |
| 3 | 2    | 11         | 13         |
| 4 | 5    | 13         | 18         |

0 + A[1]の3
 3 + A[2]の8
 11 + A[3]の2
 13 + A[4]の5
 18

## 6.4 配列の最大値

整数型: i

整数型: max

整数型の配列: A ← {2, 5, 1, 9, 8, 10, 7, 3, 6, 4}

max ← A[1]

for (i を 2 から 10 まで 1 ずつ増やす)

if (A[i] > max)

max ← A[i]

endif

endfor

表示する(max)

最初の要素 A[1] を、現在の最大値として max へ入れます。その後、残りの要素を一つずつ比較します。

max は「暫定チャンピオン」



今のチャンピオンより大きい値が現れたときだけ、max を交代する

### なぜ最大値の初期値を0にしないのか

配列の値がすべて負の数の場合、0は配列に存在しないのに最大値として残ってしまいます。最初の要素を初期値にすれば、配列内の値から最大値を選べます。

## 演習6-A 10個の合計

難易度 ★★☆☆

配列 `A` には、添字1から10までの10個の整数が入っています。合計を求めるように空欄を埋めてください。

```
整数型: i
整数型: total
整数型の配列: A

total ← [ ① ]

for (i を [ ② ] から [ ③ ] まで 1 ずつ増やす)
    total ← [ ④ ]
endfor

表示する(total)
```

AI

### AIへの質問例 | 添字と値を整理するとき

空欄の答えは表示しないでください。`i`、`A[i]`、`total` がそれぞれ何を表しているかを確認する質問を、一問ずつ出してください。

## 演習6-B 最大値の変化を追う

難易度 ★★☆☆

次の配列を使って、最大値を求める処理をトレースしてください。

$A \leftarrow \{2, 5, 1, 9, 8, 10, 7, 3, 6, 4\}$

| i   | A[i] | 比較前の max | A[i] > max | 比較後の max |
|-----|------|----------|------------|----------|
| 初期化 | 2    | —        | —          | 2        |
| 2   | 5    |          |            |          |
| 3   | 1    |          |            |          |
| 4   | 9    |          |            |          |
| 5   | 8    |          |            |          |
| 6   | 10   |          |            |          |
| 7   | 7    |          |            |          |
| 8   | 3    |          |            |          |
| 9   | 6    |          |            |          |
| 10  | 4    |          |            |          |

AI

#### AIへの質問例 | 自分のトレースを確認するとき

最終的な最大値は教えないでください。私の表で、max を更新する必要があるのに更新していない最初の行、または更新する必要がないのに更新した最初の行だけを指摘してください。

## 演習6-C 添字と値

難易度 ★☆☆

$A = \{8, 3, 6\}$ で、添字は1から始まります。最後の $x$ と $A$ の内容を求めてください。

この問題の変数は整数型です。配列の添字は1から始まります。

```
x ← A[2]
A[1] ← x
```

自分で書く： 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習6-Cについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習6-D 配列を一つずつ読む

難易度 ★☆☆

$A = \{4, 7, 2\}$ 、添字は1から始まります。表示される順番を書いてください。

この問題の変数は整数型です。配列の添字は1から始まります。

```
for (i を 1 から 3 まで 1 ずつ増やす)
  表示する(A[i])
endfor
```

自分で書く： 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習6-Dについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。



## 演習6-E 不足している商品を数える

難易度 ★★★

【実践】4商品の在庫 $A = \{2, 8, 0, 5\}$ を点検します。各商品は5個以上が目標です。不足している商品の種類数 $\text{count}$ と、補充する総個数 $\text{needed}$ を求めてください。添字は1からです。

この問題の変数は整数型です。配列の添字は1から始まります。

```
count ← 0
needed ← 0
for (i を 1 から 4 まで 1 ずつ増やす)
  if (A[i] < 5)
    count ← count + 1
    needed ← needed + (5 - A[i])
  endif
endfor
```

自分で書く：答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習6-Eについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習6-F 最初に一致する商品を探す

難易度 ★★★

【実践】商品番号A={7,4,7,9}からtarget=7を探します。位置は1から、未発見を0とします。最後のpositionを求めてください。条件のposition=0を消すと何が変わりますか。target=6の場合も教えてください。

先に確認：探索とは、目的の値を探す処理です。position(ポジション)は位置を覚える変数で、0は「未発見」の目印です。本間は先頭から一つずつ見る方法です。

この問題の変数は整数型です。配列の添字は1から始まります。

```
position ← 0
for (i を 1 から 4 まで 1 ずつ増やす)
  if (A[i] = target and position = 0)
    position ← i
  endif
endfor
```

自分で書く：答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習6-Fについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 第7章 トレースの技術

### 7.1 コードを眺めるだけでは解けない

科目Bでは、擬似言語を見て「何となく分かった」と感じて、選択肢を正しく選べないことがあります。値の変化を紙に書き、処理を実行した結果を確認する必要があります。

この作業をトレースといいます。

1回目  $\text{total} \leftarrow 0 + 1$

2回目  $\text{total} \leftarrow 1 + 2$

3回目  $\text{total} \leftarrow 3 + 3$

頭の中だけで追わない

→ 「何回目か」「計算前」「計算後」を表へ書く。トレースは、プログラムの実況中継です。

### 7.2 最初に表の列を決める

すべての変数を書く必要はありません。次を目安に列を作ります。

いつ？

$i$

何回目か

何を使う？

$A[i]$

現在の要素

どう変わる？

$\text{total}$

実行前と実行後

- 繰返しを制御する変数
- 条件式で参照する変数
- 処理によって値が更新される変数
- 配列の場合は、添字と現在参照する要素

## 例

```
整数型: i
整数型: total

total ← 0

for (i を 1 から 3 まで 1 ずつ増やす)
    total ← total + i × 2
endfor
```

| i | i × 2 | 実行前の total | 実行後の total |
|---|-------|------------|------------|
| 1 | 2     | 0          | 2          |
| 2 | 4     | 2          | 6          |
| 3 | 6     | 6          | 12         |

## 7.3 「いつの値か」を明確にする

次の二つは意味が異なります。

- 文を実行する前の値
- 文を実行した後の値

トレース表の見出しに「実行前」「実行後」と書くと、混乱を防げます。

## 7.4 条件判定も一行として記録する

選択処理や繰返しでは、処理本体だけでなく条件判定も行われます。

```
i ← 1

while (i ≤ 3)
    表示する(i)
    i ← i + 1
endwhile
```

$i$  が4になったときも、条件  $i \leq 3$  は一度判定されます。ただし、処理本体は実行されません。

| $i$ | $i \leq 3$ | 表示 | 更新後の $i$ |
|-----|------------|----|----------|
| 1   | true       | 1  | 2        |
| 2   | true       | 2  | 3        |
| 3   | true       | 3  | 4        |
| 4   | false      | —  | —        |

## 7.5 AIにトレースさせる前に

AIへ「トレース表を作って」と頼むだけでは、AIが作った表を眺めて終わってしまいます。次の順序で使います。

1. 自分で必要な列を決める
2. 最初の一回分を自分で記入する
3. AIには列の過不足、または最初の誤りだけを確認させる
4. 残りを自分で埋める
5. 最後に解説と照合する

AI

### AIへの質問例 | 表の設計を確認するとき

この擬似言語をトレースするために、私は表の列を「 $i$ 、 $A[i]$ 、total」としました。答えや完成表は表示せず、追跡に不足している列、または不要な列があれば、その理由だけを教えてください。

## 演習7-A 最初の誤りを見つける

難易度 ★★☆☆

次の処理を考えます。

整数型: i

整数型: total

total ← 1

for (i を 1 から 4 まで 1 ずつ増やす)

total ← total × i

endfor

ある受講者が次の表を作りました。最初に間違っている行を見つけ、正しい値を記入してください。

| i | 実行前の total | total × i | 実行後の total |
|---|------------|-----------|------------|
| 1 | 1          | 1         | 1          |
| 2 | 1          | 2         | 2          |
| 3 | 2          | 5         | 5          |
| 4 | 5          | 20        | 20         |

AI

#### AIへの質問例 | 自分の説明をレビューするとき

本教材の演習7-Aについてです。私は「○行目の○○が最初の誤り」と考えました。自分の表を添付するので、答えを先に示さず、どの計算を確認し直すべきかヒントを一つください。

## 演習7-B 実行前と実行後

難易度 ★☆☆

二行目の実行直前と直後のxを、それぞれ答えてください。

この問題の変数は整数型です。配列の添字は1から始まります。

```
x ← 3
x ← x + 4
```

自分で書く： 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習7-Bについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習7-C 通らない行を見分ける

難易度 ★☆☆

実行される代入文だけを順に書き、最後のyを求めてください。

この問題の変数は整数型です。配列の添字は1から始まります。

```
x ← 2
if (x > 5)
  y ← 10
else
  y ← 20
endif
```

自分で書く： 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習7-Cについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習7-D 繰返しの出口を記録する

難易度 ★★☆☆

判定時の*i*、条件の真偽、totalを表にしてください。最後の*i*とtotalはいくつですか。

この問題の変数は整数型です。配列の添字は1から始まります。

```
i ← 1
total ← 0
while (i ≤ 3)
  total ← total + i
  i ← i + 1
endwhile
```

自分で書く：答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習7-Dについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。



## 演習7-E 値引き処理の境界を検証する

難易度 ★★★

【実践】一件の注文が1000円以上なら100円引きにします。 $A = \{999, 1000, 1500\}$ の3件を処理する次のコードについて、各回のpayとtotalを表にしてください。条件を $> 1000$ に誤ると合計はいくら変わりますか。

この問題の変数は整数型です。配列の添字は1から始まります。

```
total ← 0
for (i を 1 から 3 まで 1 ずつ増やす)
  pay ← A[i]
  if (pay ≥ 1000)
    pay ← pay - 100
  endif
  total ← total + pay
endfor
```

自分で書く：答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例：本教材の演習7-Eについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習7-F 連続した記録をまとめる

難易度 ★★★

【実践】同じ番号が連続した部分を一つのまとまりとして数えます。 $A = \{2, 2, 5, 5, 2\}$ です。最初のまとまりを1とし、前の値と違ったときだけ増やす処理をトレースしてください。「異なる番号の種類数」との違いも説明してください。

先に確認:  $A[i - 1]$ は、一つ前の位置の値です。 $i = 2$ から始めるので、存在しない $A[0]$ は読みません。 $\neq$ は「等しくない」です。

この問題の変数は整数型です。配列の添字は1から始まります。

```
groups ← 1
for (i を 2 から 5 まで 1 ずつ増やす)
  if (A[i] ≠ A[i - 1])
    groups ← groups + 1
  endif
endfor
```

自分で書く: 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例: 本教材の演習7-Fについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

# 第8章 総合演習

この章では、複数の知識を組み合わせます。最初からAIへ入力せず、問題ごとに次の欄を埋めてください。

| 問題を解く前の確認欄 |       |
|------------|-------|
| 使われている変数   | ----- |
| 使われている配列   | ----- |
| 選択処理       | ----- |
| 繰り返し処理     | ----- |
| 求めている結果    | ----- |

## 演習8-A 条件に合う値の件数

難易度 ★★☆☆

配列 A には10個の整数が入っています。5以上の値がいくつあるかを数えて表示します。空欄を埋めてください。

```
整数型: i
整数型: count
整数型の配列: A

count ← [ ① ]

for ( i を 1 から 10 まで 1 ずつ増やす )
  if ( [ ② ] )
    [ ③ ]
  endif
endfor

表示する(count)
```

空欄①～③を私が埋めました。完成コードを示さず、各空欄が「初期化」「条件判定」「件数の更新」のどの役割になっているかを確認してください。誤りがあれば、最初の空欄だけを指摘してください。

## 演習8-B 最大値とその位置

難易度 ★★★

配列 **A** の最大値と、その値が入っている添字を表示します。配列には少なくとも一つの要素があり、添字は1から始まります。最大値が複数ある場合は、最初に現れる位置を表示します。

```
整数型: i
整数型: max
整数型: maxIndex
整数型の配列: A

max ← A[1]
maxIndex ← 1

for (i を 2 から Aの要素数 まで 1 ずつ増やす)
  if ([ ① ])
    max ← [ ② ]
    maxIndex ← [ ③ ]
  endif
endfor

表示する(max)
表示する(maxIndex)
```

### 考えるポイント

- 条件を  $\geq$  にすると、最大値が複数ある場合に何が起きるか
- **max** を更新するタイミングと **maxIndex** を更新するタイミング
- 繰返しを2から始める理由

## AIへの質問例 | 条件の違いを考える

$A[i] > \max$  と  $A[i] \geq \max$  の違いについて、具体的な配列を一つ使って質問形式で説明してください。最終的な空欄の答えは表示しないでください。

## 演習8-C 記号を正方形に表示する

難易度 ★★★

1以上の整数 `num` が入力されます。`num` 行、各行に `num` 個の `*` を表示し、正方形を作ります。改行せずに文字を表示する処理を 横に表示する()、改行する処理を 改行する() とします。

```

整数型: num
整数型: i
整数型: j

num ← 入力する()

for ([ ① ])
  for ([ ② ])
    [ ③ ]
  endfor
  [ ④ ]
endfor

```

`num` が3の場合の出力:

```

* * *
* * *
* * *

```

## AIへの質問例 | 入れ子の繰返し

完成した擬似言語は示さないでください。外側の繰返しと内側の繰返しが、それぞれ「行数」と「一行の文字数」のどちらを担当するか、私が説明できるように質問してください。

## 演習8-D 条件付きの更新

難易度 ★☆☆

$A = \{2, 6\}$ です。最後のresultを教えてください。

この問題の変数は整数型です。配列の添字は1から始まります。

```
result ← A[1]
if (A[2] > result)
  result ← A[2]
endif
```

自分で書く: 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例: 本教材の演習8-Dについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習8-E 二つの値の合計

難易度 ★☆☆

$A = \{3, 5\}$ です。最後のtotalを教えてください。

この問題の変数は整数型です。配列の添字は1から始まります。

```
total ← 0
for (i を 1 から 2 まで 1 ずつ増やす)
  total ← total + A[i]
endfor
```

自分で書く: 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例: 本教材の演習8-Eについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

# 演習8-F 条件を満たした値だけ合計する

難易度 ★★☆☆

A = {3, 8, 5, 1}から5以上の値だけを集計します。count、total、averageを求めてください。averageは実数型です。この配列では対象が必ず一つ以上あります。

整数型: count, total, i 実数型: average 整数型の配列: A

```
count ← 0
total ← 0
for (i を 1 から 4 まで 1 ずつ増やす)
  if (A[i] ≥ 5)
    count ← count + 1
    total ← total + A[i]
  endif
endfor
average ← total ÷ count
```

自分で書く: 答えだけでなく、途中の値・条件の真偽・その理由をノートに残してください。

研修中の質問例: 本教材の演習8-Fについてです。私は〇〇と考えました。答えはまだ示さず、次に確認することを一つ教えてください。

## 演習後の振り返り

| 振り返る項目          | 記入欄 |
|-----------------|-----|
| 最初に分からなかったこと    |     |
| 自分で試したこと        |     |
| AIへ入力した質問       |     |
| AIから得たヒント       |     |
| AIの説明を確認した方法    |     |
| 次に同じ問題を解くときの注意点 |     |

## 解答・解説

---

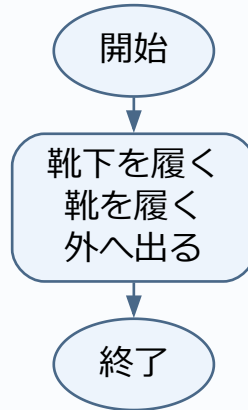
自分の答えを書いてから確認します。文章、値の確認表、フローチャートを対応させて読みましょう。図では読みやすさのため、連続する代入を一つの箱にまとめることがあります。



## 演習1-A 解答・解説

靴下→靴→外へ出る、の順です。上から一つずつ進む「順次」です。

演習1-Aのフローチャート

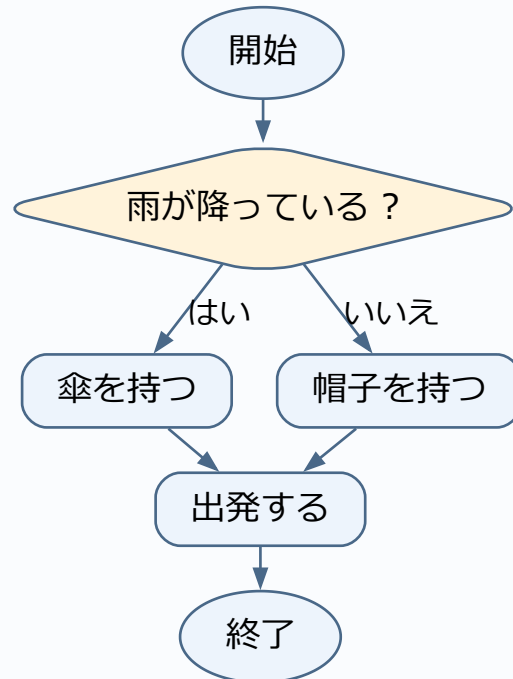


ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

## 演習1-B 解答・解説

雨の日は「傘を持つ」へ進み、「帽子を持つ」は実行しません。条件で道を分ける「選択」です。

演習1-Bのフローチャート

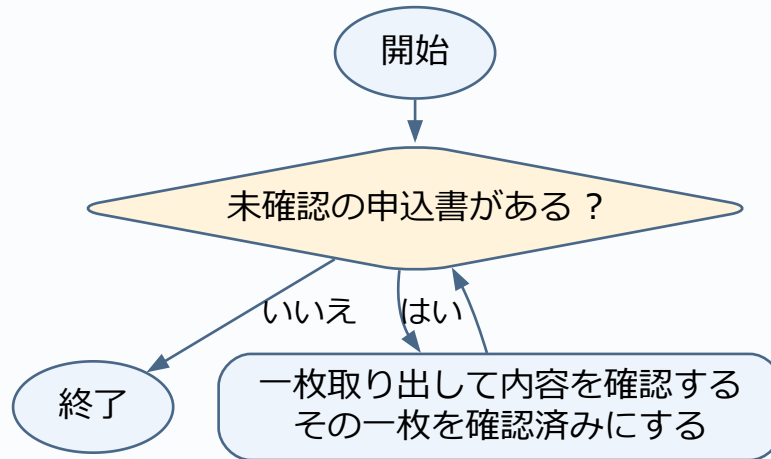


ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

## 演習1-C 解答・解説

3枚なら3回、0枚なら0回です。「繰り返し」で、毎回未確認の枚数が減るため終わります。

演習1-Cのフローチャート

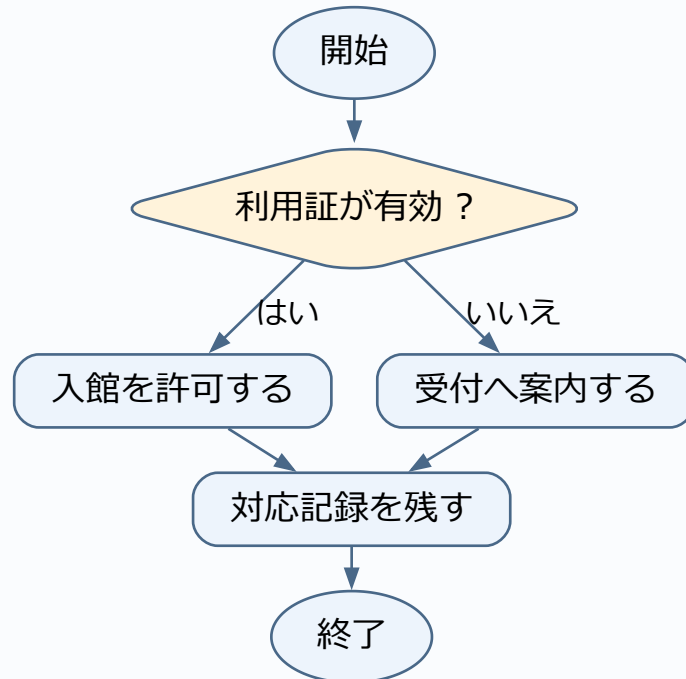


ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

## 演習1-D 解答・解説

無効だった人の対応記録が残らなくなります。分岐が合流した後に記録を置けば、どちらの場合も実行されます。

演習1-Dのフローチャート

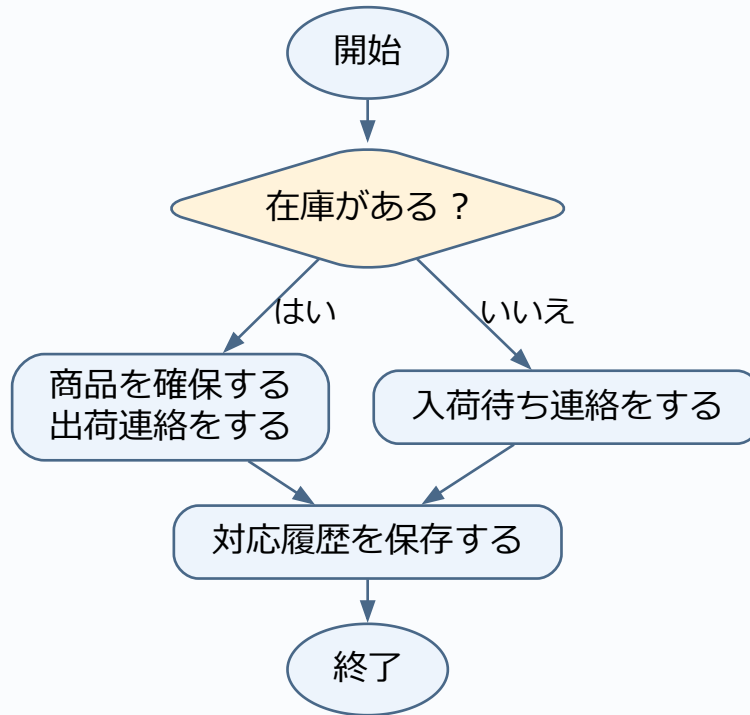


ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

## 演習1-E 解答・解説

在庫なし→入荷待ち連絡→履歴保存です。出荷連絡はしません。「最後に必ず」の処理は分岐の外に置きます。

演習1-Eのフローチャート

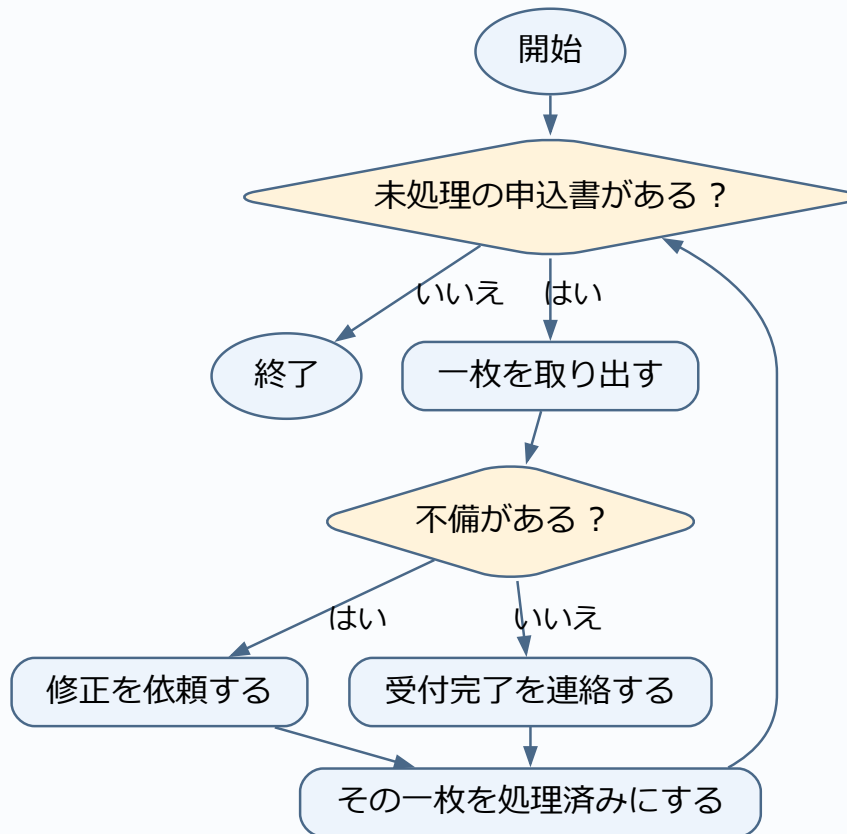


ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

## 演習1-F 解答・解説

繰返しの中に選択を置きます。処理済みにしないと、同じ申込書が未処理のまま残り、終わらないおそれがあります。図では両方の連絡から「処理済み」へ合流します。

演習1-Fのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

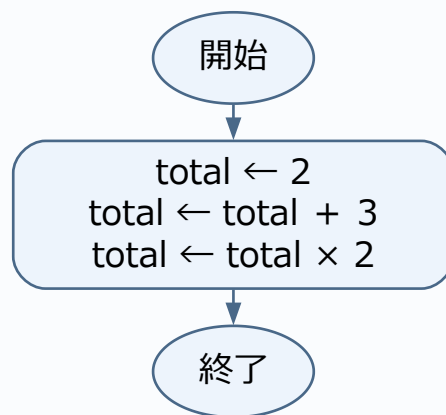
## 演習2-A 解答・解説

値は2→5→10です。右辺を現在の値で計算してから、左辺を上書きします。

値の確認(各処理の実行後)

| 実行した処理                         | その直後の値                |
|--------------------------------|-----------------------|
| <code>total ← 2</code>         | <code>total=2</code>  |
| <code>total ← total + 3</code> | <code>total=5</code>  |
| <code>total ← total × 2</code> | <code>total=10</code> |

演習2-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

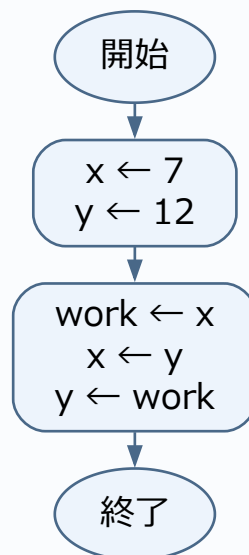
## 演習2-B 解答・解説

① x、② y、③ work。workに元のxを残すので、xを上書きしても7を取り戻せます。最終値はx=12、y=7です。

値の確認(各処理の実行後)

| 実行した処理              | その直後の値               |
|---------------------|----------------------|
| $x \leftarrow 7$    | $x=7$                |
| $y \leftarrow 12$   | $x=7, y=12$          |
| $work \leftarrow x$ | $x=7, y=12, work=7$  |
| $x \leftarrow y$    | $x=12, y=12, work=7$ |
| $y \leftarrow work$ | $x=12, y=7, work=7$  |

### 演習2-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。



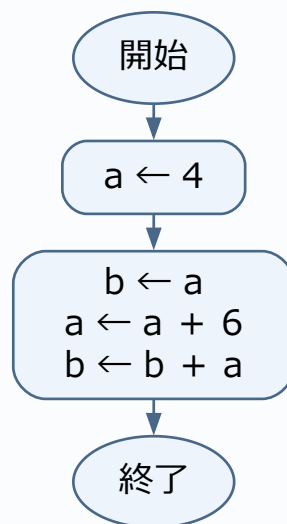
## 演習2-C 解答・解説

$a=10$ 、 $b=14$ 。 $b$ へコピーした4は、 $a$ を10に変えても自動では変わりません。最後に $4+10$ を計算します。

値の確認(各処理の実行後)

| 実行した処理               | その直後の値          |
|----------------------|-----------------|
| $a \leftarrow 4$     | $a=4$           |
| $b \leftarrow a$     | $a=4$ 、 $b=4$   |
| $a \leftarrow a + 6$ | $a=10$ 、 $b=4$  |
| $b \leftarrow b + a$ | $a=10$ 、 $b=14$ |

### 演習2-Cのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

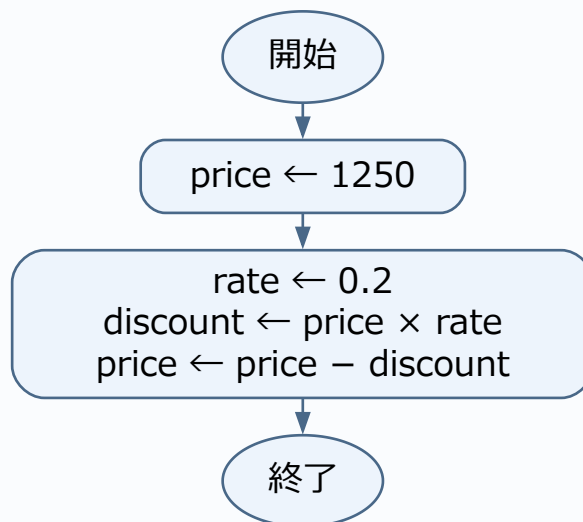
## 演習2-D 解答・解説

discount=250、price=1000です。0.2を掛けると元の金額の20%になります。割引額と支払金額を区別します。

値の確認(各処理の実行後)

| 実行した処理                   | その直後の値                           |
|--------------------------|----------------------------------|
| price ← 1250             | price=1250                       |
| rate ← 0.2               | price=1250、rate=0.2              |
| discount ← price × rate  | price=1250、rate=0.2、discount=250 |
| price ← price - discount | price=1000、rate=0.2、discount=250 |

### 演習2-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

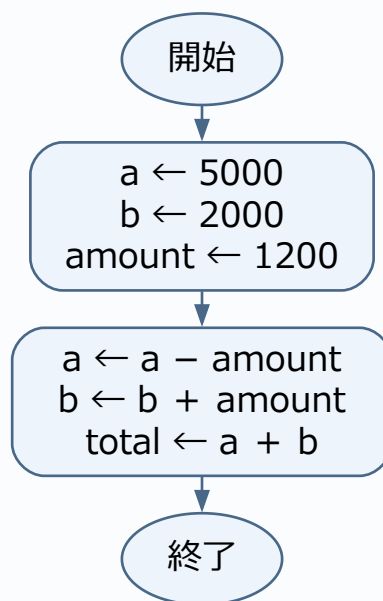
## 演習2-E 解答・解説

$a=3800$ 、 $b=3200$ 、 $total=7000$ 。片方から引いた1200をもう片方へ足すため、合計は移動前と同じです。

値の確認(各処理の実行後)

| 実行した処理                    | その直後の値                                             |
|---------------------------|----------------------------------------------------|
| $a \leftarrow 5000$       | $a=5000$                                           |
| $b \leftarrow 2000$       | $a=5000$ 、 $b=2000$                                |
| $amount \leftarrow 1200$  | $a=5000$ 、 $b=2000$ 、 $amount=1200$                |
| $a \leftarrow a - amount$ | $a=3800$ 、 $b=2000$ 、 $amount=1200$                |
| $b \leftarrow b + amount$ | $a=3800$ 、 $b=3200$ 、 $amount=1200$                |
| $total \leftarrow a + b$  | $a=3800$ 、 $b=3200$ 、 $amount=1200$ 、 $total=7000$ |

演習2-Eのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

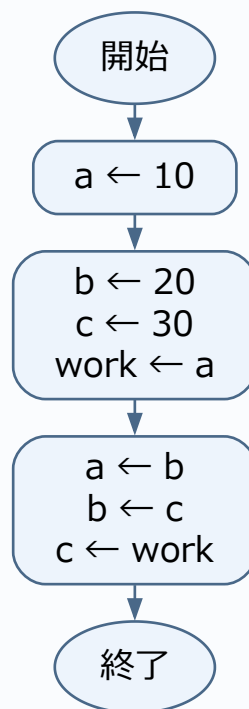
## 演習2-F 解答・解説

$a=20$ 、 $b=30$ 、 $c=10$ です。最後を $c \leftarrow a$ にすると、既に $a$ が20なので $c$ も20になり、元の10が失われます。

値の確認(各処理の実行後)

| 実行した処理              | その直後の値                               |
|---------------------|--------------------------------------|
| $a \leftarrow 10$   | $a=10$                               |
| $b \leftarrow 20$   | $a=10$ 、 $b=20$                      |
| $c \leftarrow 30$   | $a=10$ 、 $b=20$ 、 $c=30$             |
| $work \leftarrow a$ | $a=10$ 、 $b=20$ 、 $c=30$ 、 $work=10$ |
| $a \leftarrow b$    | $a=20$ 、 $b=20$ 、 $c=30$ 、 $work=10$ |
| $b \leftarrow c$    | $a=20$ 、 $b=30$ 、 $c=30$ 、 $work=10$ |
| $c \leftarrow work$ | $a=20$ 、 $b=30$ 、 $c=10$ 、 $work=10$ |

### 演習2-Fのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

公開問題への接続:変数の退避と上書き。本演習は研修用のオリジナル問題であり、リンク先の問題・正解と同一ではありません。

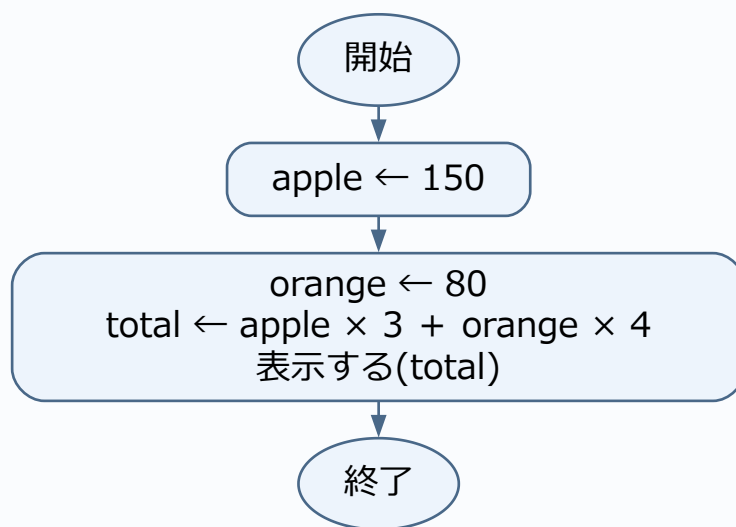
## 演習3-A 解答・解説

①150、②80、③ $\text{apple} \times 3 + \text{orange} \times 4$ 。450+320=770円です。

値の確認(各処理の実行後)

| 実行した処理                                                                   | その直後の値                                                 |
|--------------------------------------------------------------------------|--------------------------------------------------------|
| $\text{apple} \leftarrow 150$                                            | $\text{apple}=150$                                     |
| $\text{orange} \leftarrow 80$                                            | $\text{apple}=150, \text{orange}=80$                   |
| $\text{total} \leftarrow \text{apple} \times 3 + \text{orange} \times 4$ | $\text{apple}=150, \text{orange}=80, \text{total}=770$ |

演習3-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

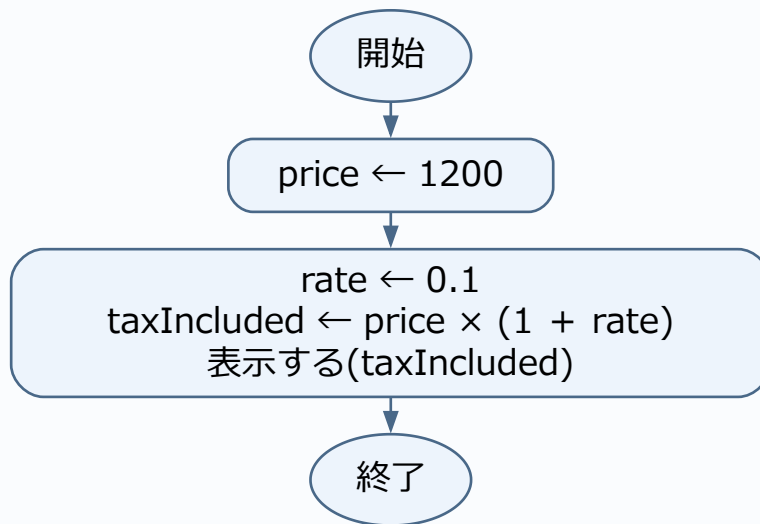
## 演習3-B 解答・解説

$\text{taxIncluded} \leftarrow \text{price} \times (1 + \text{rate})$ 、表示する( $\text{taxIncluded}$ )。税込価格は1320です。税額120だけでなく、本体価格1200も含めます。

値の確認(各処理の実行後)

| 実行した処理                                                                | その直後の値                                                              |
|-----------------------------------------------------------------------|---------------------------------------------------------------------|
| $\text{price} \leftarrow 1200$                                        | $\text{price}=1200$                                                 |
| $\text{rate} \leftarrow 0.1$                                          | $\text{price}=1200$ 、 $\text{rate}=0.1$                             |
| $\text{taxIncluded} \leftarrow \text{price} \times (1 + \text{rate})$ | $\text{price}=1200$ 、 $\text{rate}=0.1$ 、 $\text{taxIncluded}=1320$ |

### 演習3-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

## 演習3-C 解答・解説

表示は600。入力は単価と個数、処理は掛け算、出力は合計金額の表示です。

値の確認(各処理の実行後)

| 実行した処理                   | その直後の値                               |
|--------------------------|--------------------------------------|
| price ← 200              | price = 200                          |
| quantity ← 3             | price = 200、quantity = 3             |
| total ← price × quantity | price = 200、quantity = 3、total = 600 |

演習3-Cのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

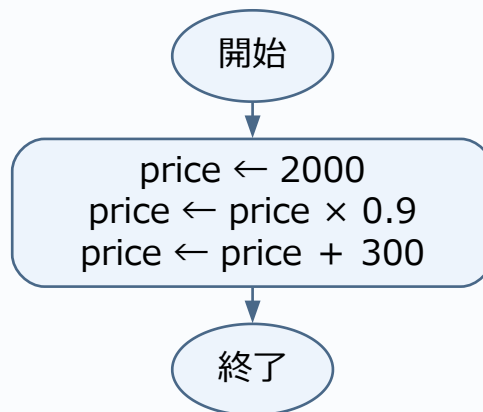
## 演習3-D 解答・解説

price=2100。先に送料を足すと $2300 \times 0.9 = 2070$ です。後者では送料まで割引対象になるため30円違います。

値の確認(各処理の実行後)

| 実行した処理              | その直後の値     |
|---------------------|------------|
| price ← 2000        | price=2000 |
| price ← price × 0.9 | price=1800 |
| price ← price + 300 | price=2100 |

### 演習3-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。



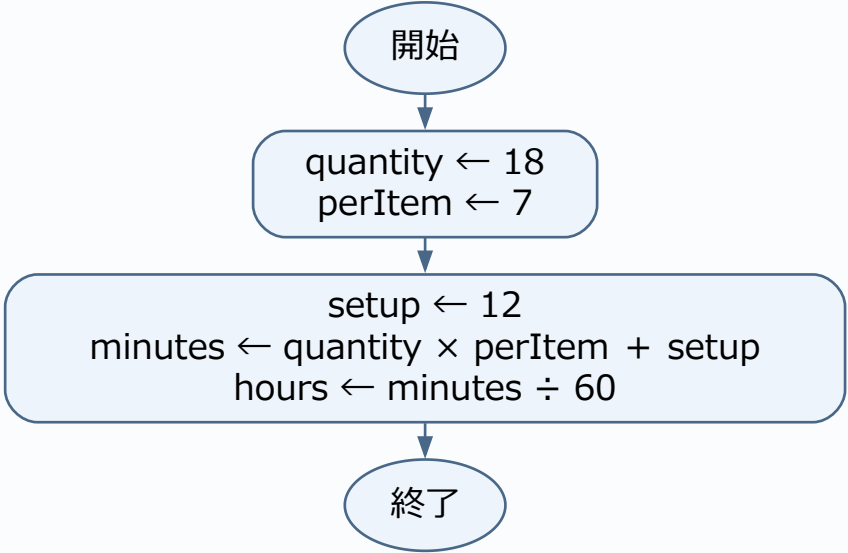
# 演習3-E 解答・解説

minutes=138分、hours=2.3時間。準備は全体で一回なので、製造時間126分に12分だけ加えます。

値の確認(各処理の実行後)

| 実行した処理                               | その直後の値                                               |
|--------------------------------------|------------------------------------------------------|
| quantity ← 18                        | quantity=18                                          |
| perItem ← 7                          | quantity=18、perItem=7                                |
| setup ← 12                           | quantity=18、perItem=7、setup=12                       |
| minutes ← quantity × perItem + setup | quantity=18、perItem=7、setup=12、minutes=138           |
| hours ← minutes ÷ 60                 | quantity=18、perItem=7、setup=12、minutes=138、hours=2.3 |

演習3-Eのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

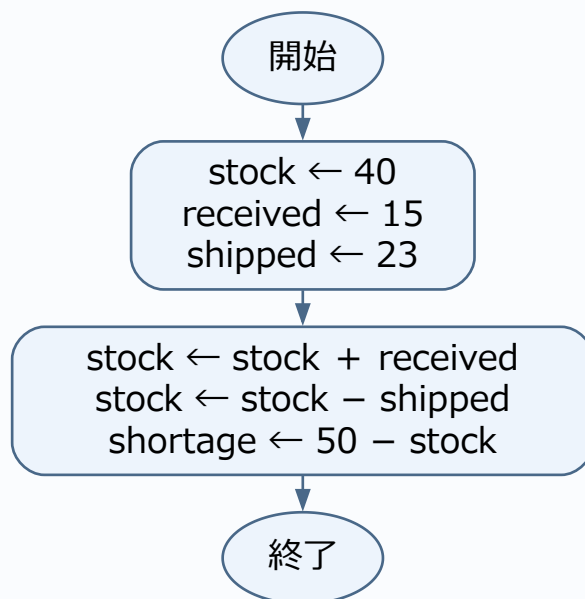
## 演習3-F 解答・解説

stock=32、shortage=18。入荷後は55、出荷後は32です。不足数は、出荷後の値を使って50-32と計算します。

値の確認(各処理の実行後)

| 実行した処理                   | その直後の値                                      |
|--------------------------|---------------------------------------------|
| stock ← 40               | stock=40                                    |
| received ← 15            | stock=40、received=15                        |
| shipped ← 23             | stock=40、received=15、shipped=23             |
| stock ← stock + received | stock=55、received=15、shipped=23             |
| stock ← stock - shipped  | stock=32、received=15、shipped=23             |
| shortage ← 50 - stock    | stock=32、received=15、shipped=23、shortage=18 |

演習3-Fのフローチャート



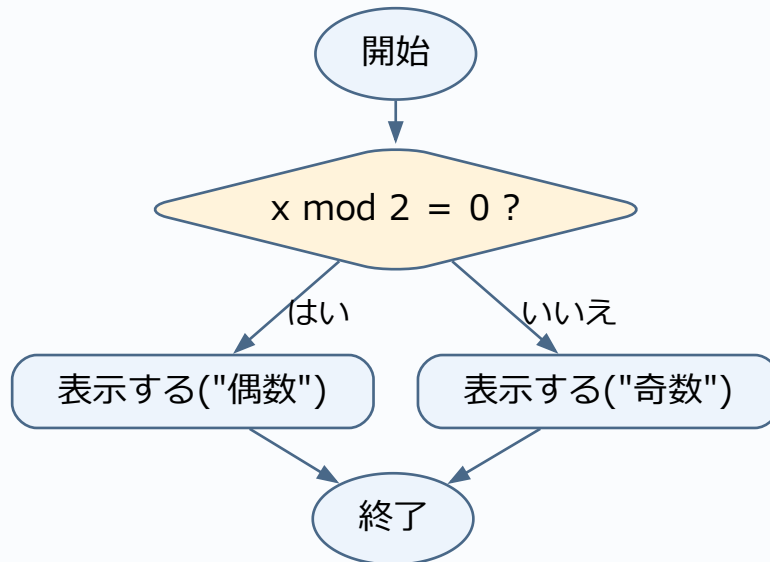
ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

## 演習4-A 解答・解説

条件は $x \bmod 2 = 0$ です。余りが0の側だけが偶数になります。 $x=8$ なら偶数、 $x=7$ なら奇数です。

表の確認に使う入力: $x=8$ 。

演習4-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

## 演習4-B 解答・解説

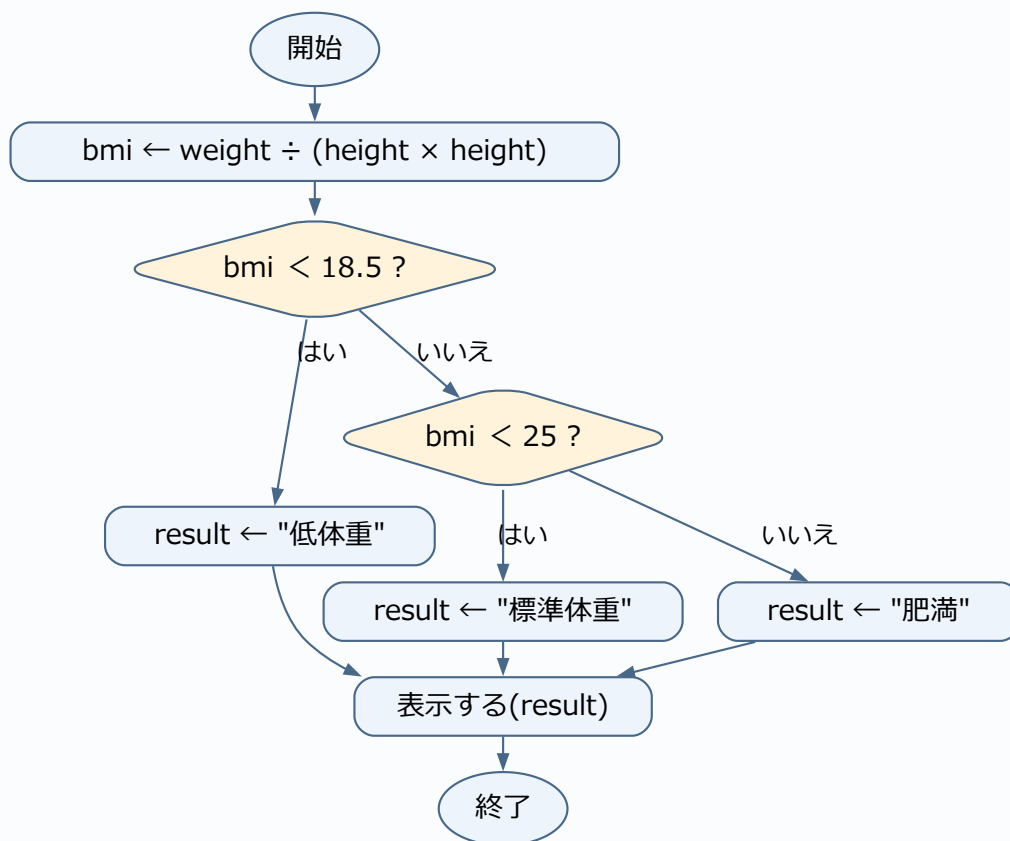
①  $\text{bmi} < 18.5$ 、②  $\text{bmi} < 25$ 。18.49は低体重、18.50と24.99は標準体重、25.00は肥満です。二つ目へ来た時点で18.5以上は確定しています。

表の確認に使う入力:  $\text{height}=1$ 、 $\text{weight}=18.5$ 。

値の確認(各処理の実行後)

| 実行した処理                                                                          | その直後の値                                          |
|---------------------------------------------------------------------------------|-------------------------------------------------|
| $\text{bmi} \leftarrow \text{weight} \div (\text{height} \times \text{height})$ | $\text{bmi}=18.5$                               |
| $\text{result} \leftarrow \text{"標準体重"}$                                        | $\text{bmi}=18.5$ 、 $\text{result}=\text{標準体重}$ |

演習4-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

## 演習4-C 解答・解説

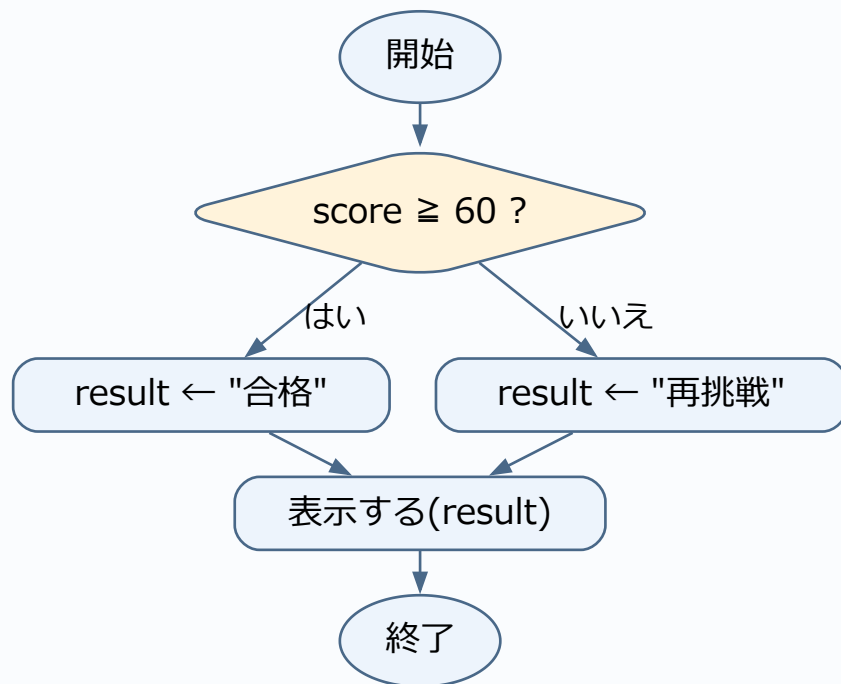
59は再挑戦、60は合格です。「以上」は境目の60自身も含みます。

表の確認に使う入力:score=59。

値の確認(各処理の実行後)

| 実行した処理         | その直後の値     |
|----------------|------------|
| result ← "再挑戦" | result=再挑戦 |

演習4-Cのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

## 演習4-D 解答・解説

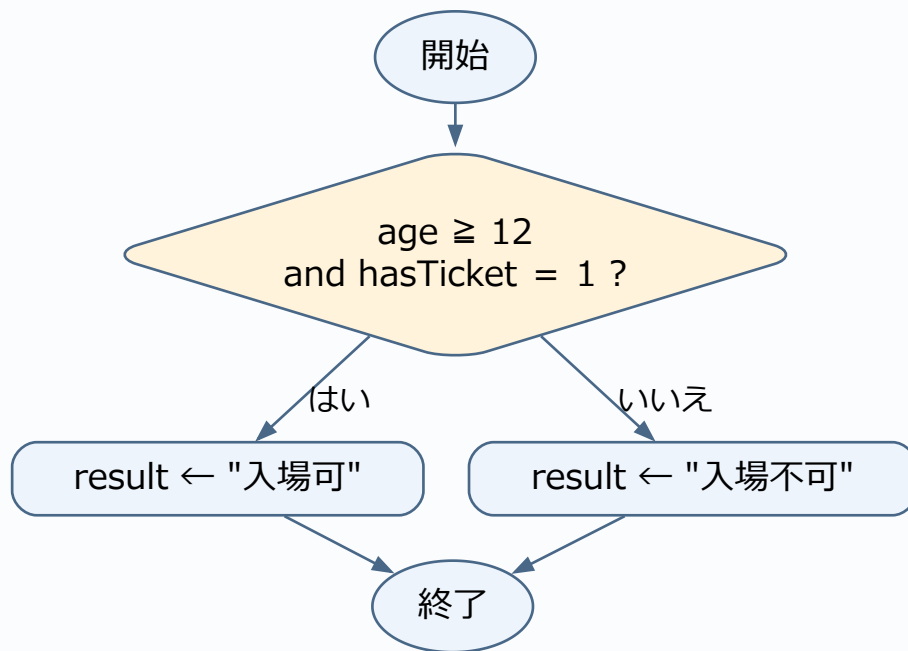
順に入場可、入場不可、入場不可です。andは両方が真のときだけ真になります。

表の確認に使う入力: age=12、hasTicket=1。

値の確認(各処理の実行後)

| 実行した処理         | その直後の値     |
|----------------|------------|
| result ← "入場可" | result=入場可 |

演習4-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

# 演習4-E 解答・解説

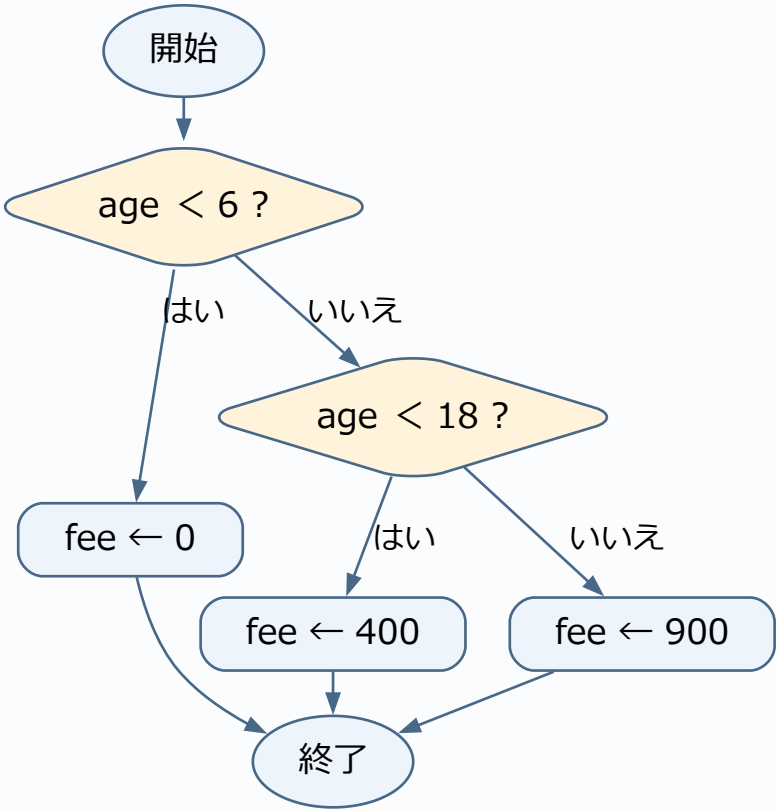
順に0、400、400、900円。≤18にすると18歳が400円となり誤りです。境目の直前と境目自身を必ず確認します。

表の確認に使う入力:age=5。

値の確認(各処理の実行後)

| 実行した処理  | その直後の値 |
|---------|--------|
| fee ← 0 | fee=0  |

演習4-Eのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

公開問題への接続:条件分岐の読み取り。本演習は研修用のオリジナル問題であり、リンク先の問題・正解と同一ではありません。

## 演習4-F 解答・解説

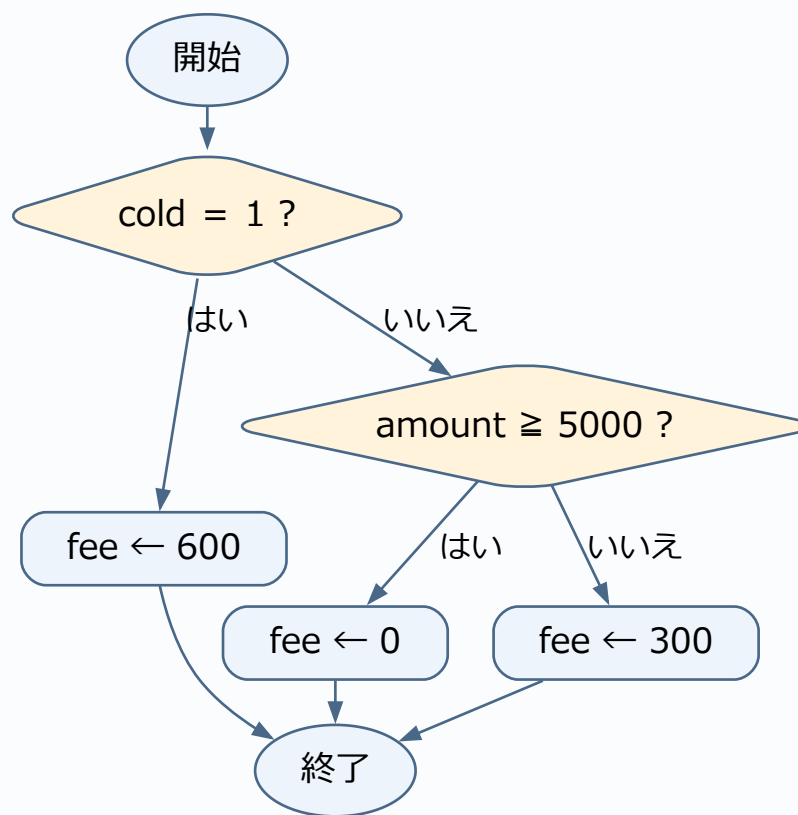
順に600、0、300円。冷蔵条件を優先します。「5000円以上なら無料」だけを先に適用すると、冷蔵便まで無料にしてしまいます。

表の確認に使う入力: cold=1、amount=6000。

値の確認(各処理の実行後)

| 実行した処理    | その直後の値  |
|-----------|---------|
| fee ← 600 | fee=600 |

演習4-Fのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。



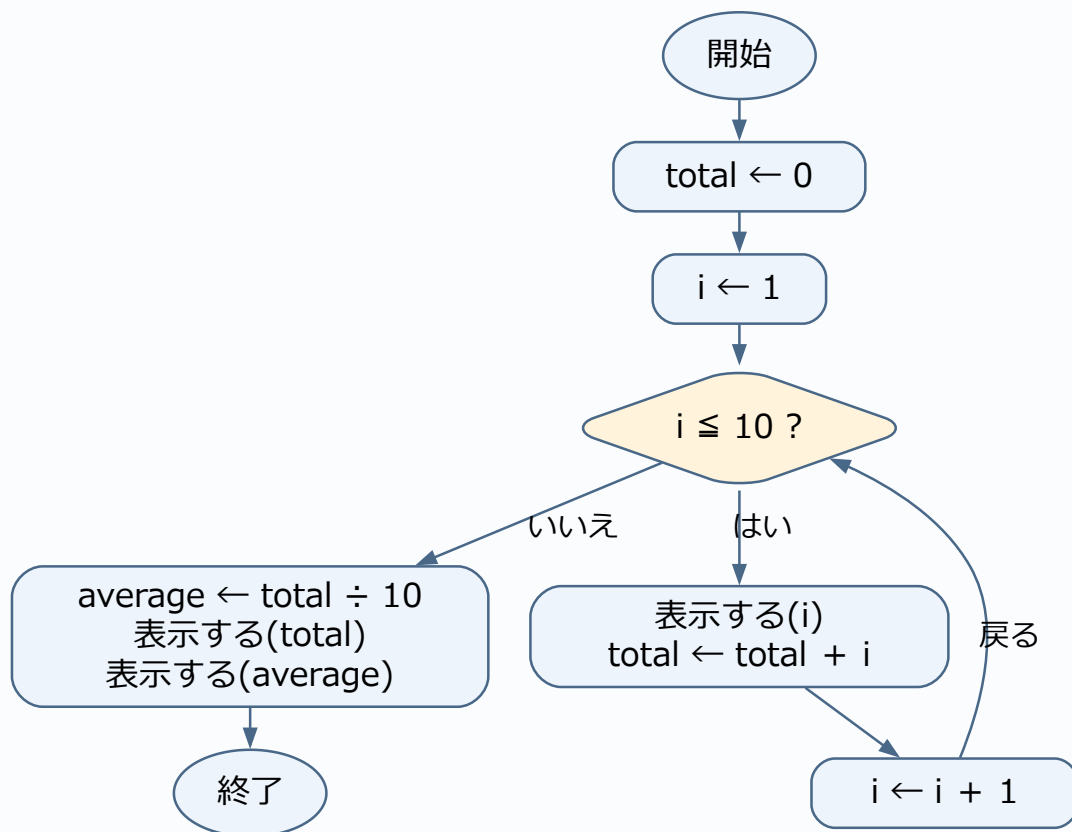
## 演習5-A 解答・解説

①0、②1、③10、④total + i、⑤total ÷ 10。合計55、平均5.5です。図はforを初期化・条件判定・更新に分解しています。

値の確認(各処理の実行後)

| 実行した処理               | その直後の値                    |
|----------------------|---------------------------|
| total ← 0            | total=0                   |
| i=1の回を終了             | total=1、i=1               |
| i=2の回を終了             | total=3、i=2               |
| …(中間の反復を省略)          |                           |
| i=9の回を終了             | total=45、i=9              |
| i=10の回を終了            | total=55、i=10             |
| average ← total ÷ 10 | total=55、i=11、average=5.5 |

演習5-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

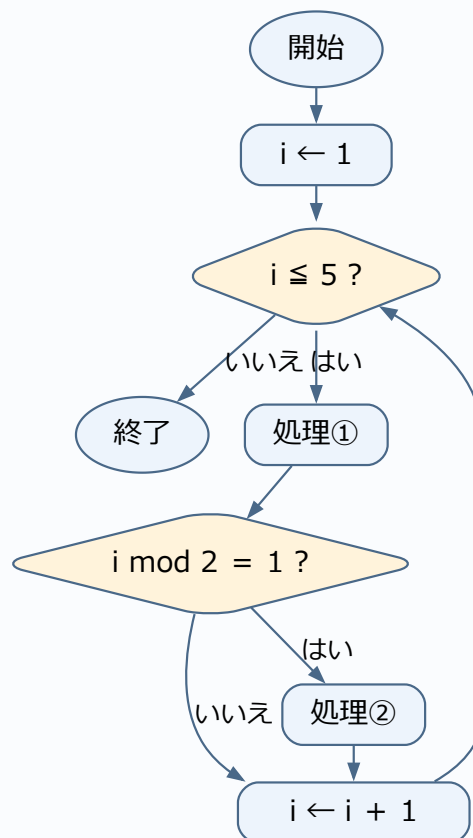
## 演習5-B 解答・解説

処理①は5回、処理②は $i=1, 3, 5$ の3回。終了時 $i=6$ です。 $i \leq 5$ の判定自体は、最後の偽を含め6回行います。

値の確認(各処理の実行後)

| 実行した処理           | その直後の値 |
|------------------|--------|
| $i \leftarrow 1$ | $i=1$  |
| 繰返し一回分を終了        | $i=2$  |
| 繰返し一回分を終了        | $i=3$  |
| 繰返し一回分を終了        | $i=4$  |
| 繰返し一回分を終了        | $i=5$  |
| 繰返し一回分を終了        | $i=6$  |

演習5-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

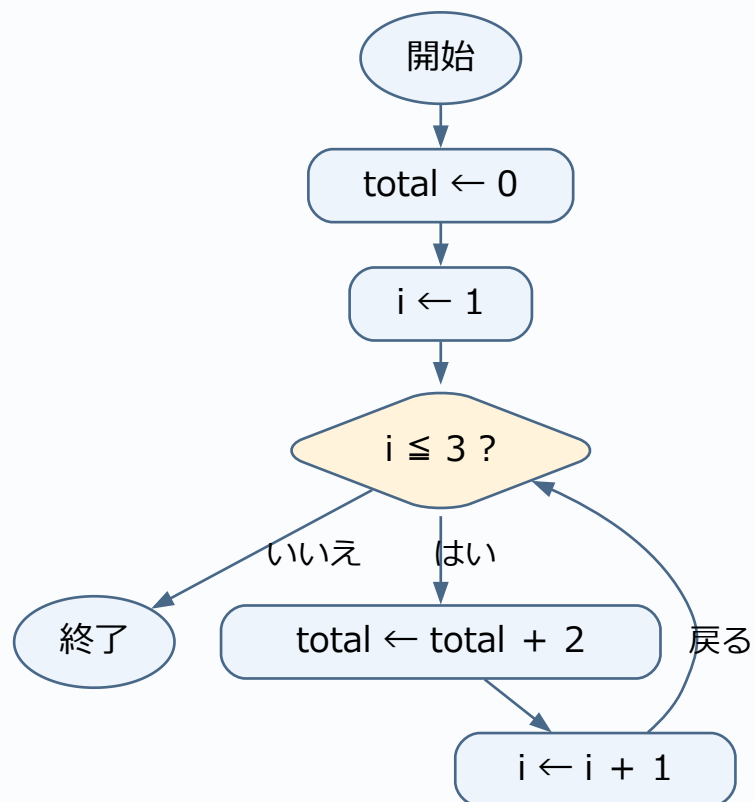
## 演習5-C 解答・解説

totalは0→2→4→6です。i=1、2、3の計3回、2を足します。

値の確認(各処理の実行後)

| 実行した処理    | その直後の値      |
|-----------|-------------|
| total ← 0 | total=0     |
| i=1の回を終了  | total=2、i=1 |
| i=2の回を終了  | total=4、i=2 |
| i=3の回を終了  | total=6、i=3 |

演習5-Cのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

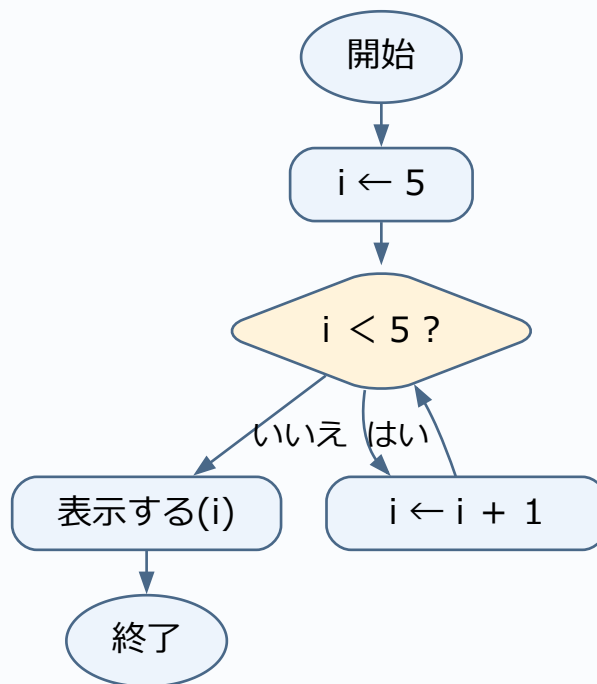
## 演習5-D 解答・解説

表示は5、内部は0回です。whileは、実行する前に条件を確かめます。最初から偽なら一度も入りません。

値の確認(各処理の実行後)

| 実行した処理           | その直後の値  |
|------------------|---------|
| $i \leftarrow 5$ | $i = 5$ |

演習5-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

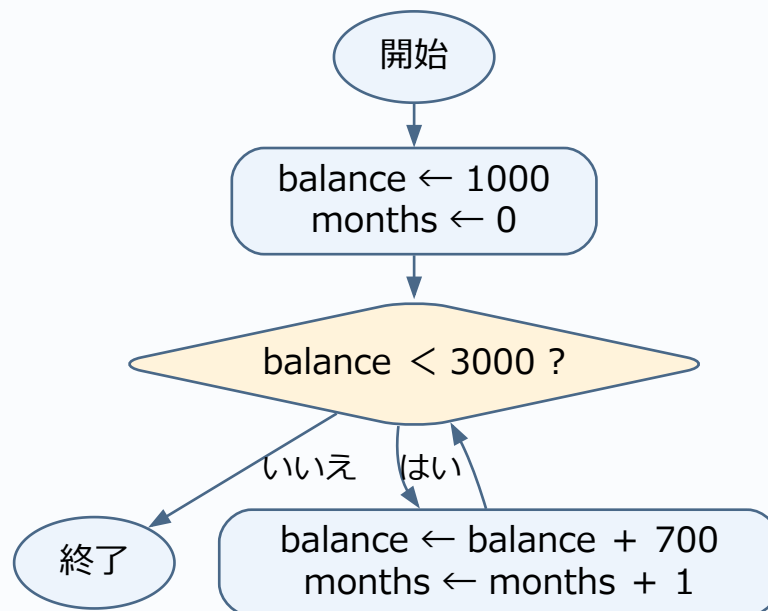
## 演習5-E 解答・解説

3か月、3100円です。1000→1700→2400→3100と変化し、条件は4回判定します。最後は目標を超えても停止します。

値の確認(各処理の実行後)

| 実行した処理         | その直後の値                    |
|----------------|---------------------------|
| balance ← 1000 | balance = 1000            |
| 繰返し一回分を終了      | balance = 1700、months = 1 |
| 繰返し一回分を終了      | balance = 2400、months = 2 |
| 繰返し一回分を終了      | balance = 3100、months = 3 |

演習5-Eのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

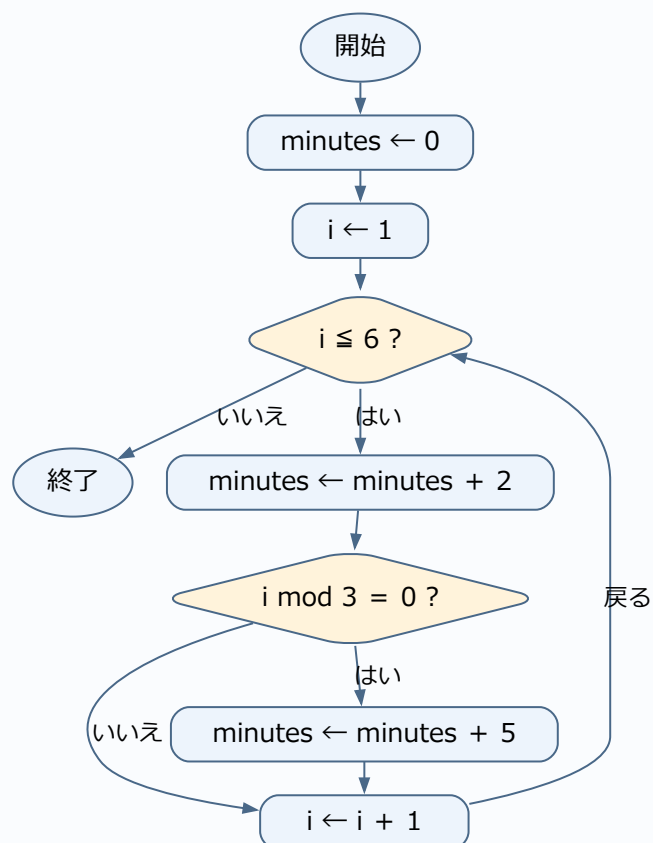
## 演習5-F 解答・解説

22分です。通常処理 $6 \times 2 = 12$ 分に、追加点検 $2 \times 5 = 10$ 分を加えます。追加点検の日でも通常の2分は必要です。

値の確認(各処理の実行後)

| 実行した処理                 | その直後の値         |
|------------------------|----------------|
| minutes $\leftarrow$ 0 | minutes=0      |
| i=1の回を終了               | minutes=2、i=1  |
| i=2の回を終了               | minutes=4、i=2  |
| i=3の回を終了               | minutes=11、i=3 |
| i=4の回を終了               | minutes=13、i=4 |
| i=5の回を終了               | minutes=15、i=5 |
| i=6の回を終了               | minutes=22、i=6 |

演習5-Fのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

# 演習6-A 解答・解説

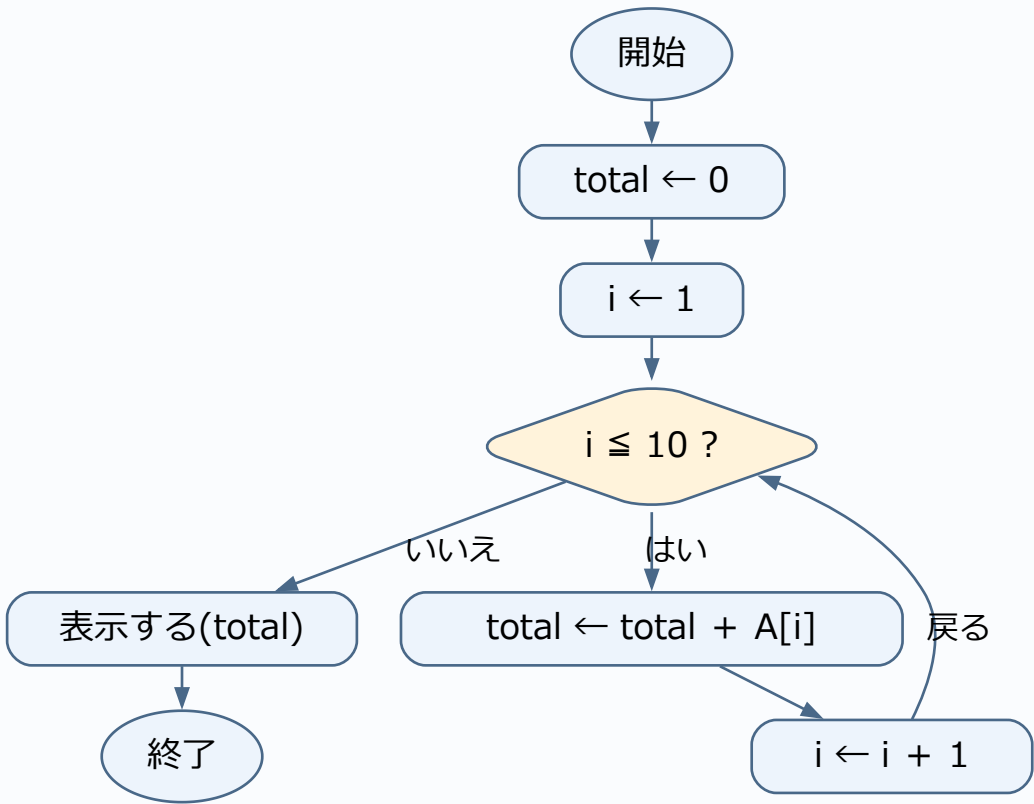
①0、②1、③10、④total + A[i]。iは位置、A[i]はその位置の値です。例えばAが1～10なら合計55。実際の合計は与えられた配列で変わります。

表の確認に使う入力:A={1,2,3,4,5,6,7,8,9,10}。

値の確認(各処理の実行後)

| 実行した処理        | その直後の値        |
|---------------|---------------|
| total ← 0     | total=0       |
| i=1の回を終了      | total=1、i=1   |
| i=2の回を終了      | total=3、i=2   |
| ...(中間の反復を省略) |               |
| i=8の回を終了      | total=36、i=8  |
| i=9の回を終了      | total=45、i=9  |
| i=10の回を終了     | total=55、i=10 |

演習6-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

## 演習6-B 解答・解説

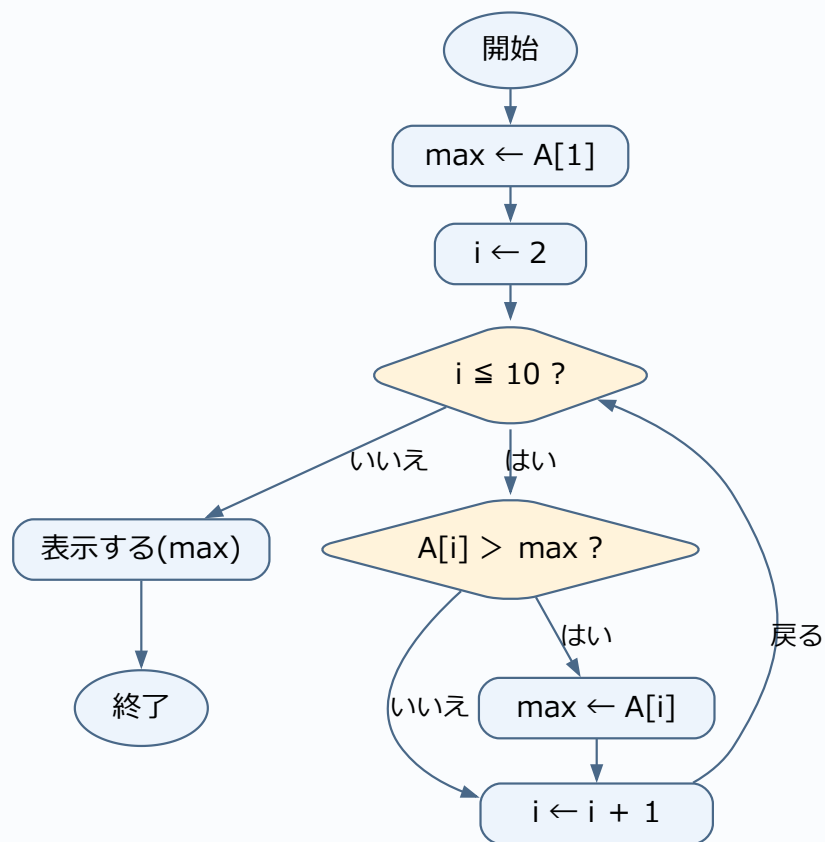
maxは初期値2から、5→5→9→9→10→10→10→10→10。現在の最大値より大きい場合だけ更新します。

表の確認に使う入力:A={2,5,1,9,8,10,7,3,6,4}。

値の確認(各処理の実行後)

| 実行した処理                       | その直後の値                |
|------------------------------|-----------------------|
| $\text{max} \leftarrow A[1]$ | $\text{max}=2$        |
| i=2の回を終了                     | $\text{max}=5, i=2$   |
| i=3の回を終了                     | $\text{max}=5, i=3$   |
| ...(中間の反復を省略)                |                       |
| i=8の回を終了                     | $\text{max}=10, i=8$  |
| i=9の回を終了                     | $\text{max}=10, i=9$  |
| i=10の回を終了                    | $\text{max}=10, i=10$ |

演習6-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。



# 演習6-C 解答・解説

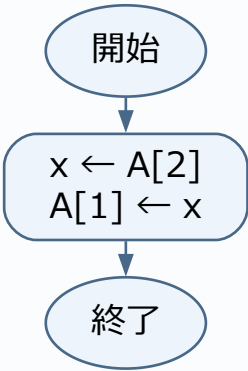
$x=3$ 、 $A=\{3,3,6\}$ です。 $A[2]$ の2は値ではなく位置を指定しています。

表の確認に使う入力: $A=\{8,3,6\}$ 。

値の確認(各処理の実行後)

| 実行した処理              | その直後の値                  |
|---------------------|-------------------------|
| $x \leftarrow A[2]$ | $A=\{8, 3, 6\}$ 、 $x=3$ |
| $A[1] \leftarrow x$ | $A=\{3, 3, 6\}$ 、 $x=3$ |

演習6-Cのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

## 演習6-D 解答・解説

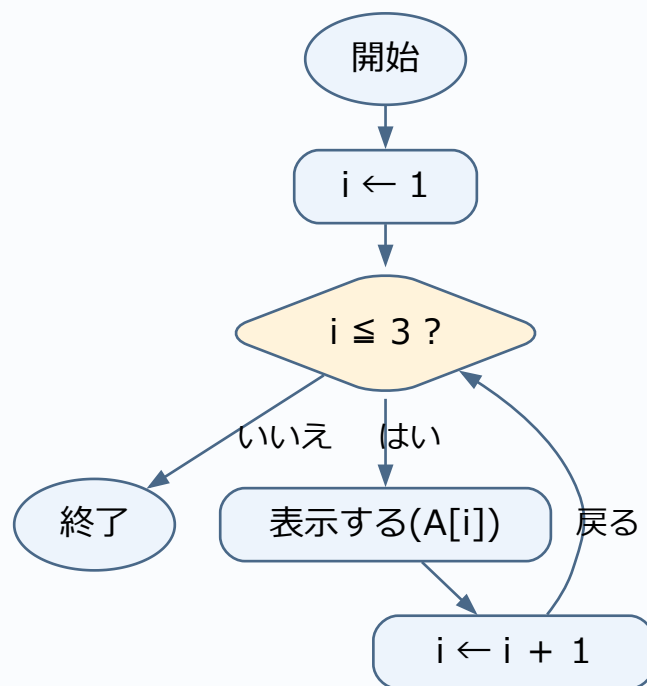
4、7、2の順です。表示するのは添字1、2、3ではなく、その位置に入っている値です。

表の確認に使う入力:  $A = \{4, 7, 2\}$ 。

値の確認(各処理の実行後)

| 実行した処理   | その直後の値 |
|----------|--------|
| i=1の回を終了 | i=1    |
| i=2の回を終了 | i=2    |
| i=3の回を終了 | i=3    |

演習6-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

## 演習6-E 解答・解説

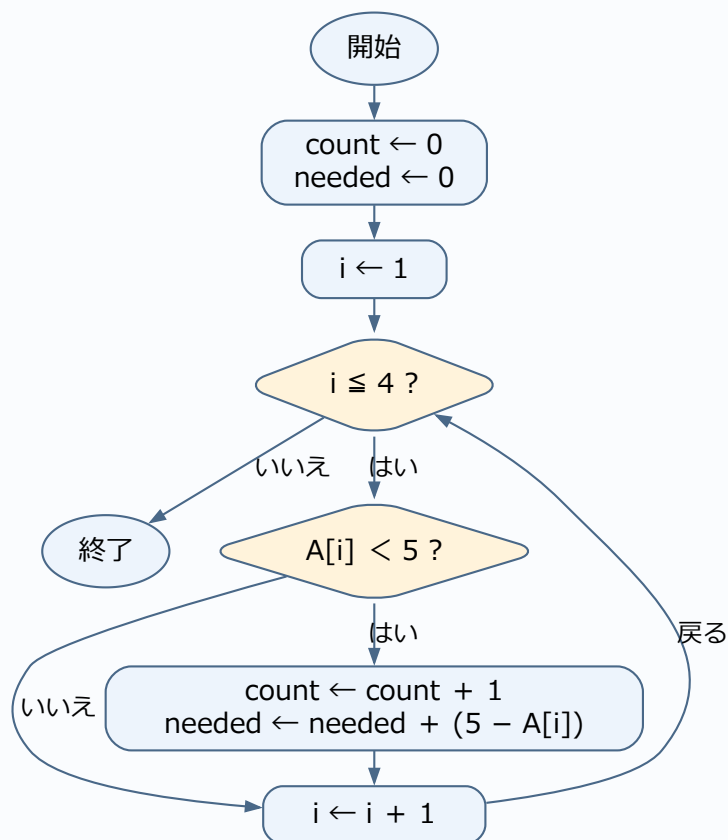
count=2種類、needed=8個。1番に3個、3番に5個補充します。5個ちょうど4番は不足していません。

表の確認に使う入力:  $A = \{2, 8, 0, 5\}$ 。

値の確認(各処理の実行後)

| 実行した処理               | その直後の値               |
|----------------------|----------------------|
| count $\leftarrow$ 0 | count=0              |
| i=1の回を終了             | count=1、needed=3、i=1 |
| i=2の回を終了             | count=1、needed=3、i=2 |
| i=3の回を終了             | count=2、needed=8、i=3 |
| i=4の回を終了             | count=2、needed=8、i=4 |

演習6-Eのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

## 演習6-F 解答・解説

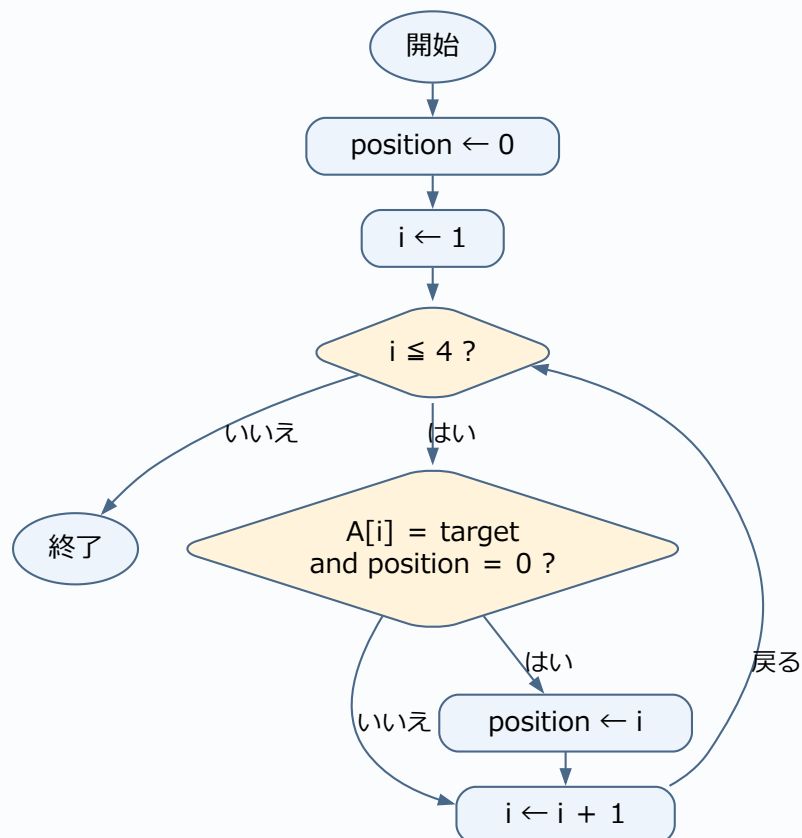
target=7ならposition=1。既に見つかったら更新しないため最初の位置が残ります。position=0を消すと最後の一致位置3になります。target=6なら未発見の0です。

表の確認に使う入力: A={7,4,7,9}, target=7。

値の確認(各処理の実行後)

| 実行した処理       | その直後の値          |
|--------------|-----------------|
| position ← 0 | position=0      |
| i=1の回を終了     | position=1, i=1 |
| i=2の回を終了     | position=1, i=2 |
| i=3の回を終了     | position=1, i=3 |
| i=4の回を終了     | position=1, i=4 |

演習6-Fのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

公開問題への接続: 探索の考え方(公開問題は二分探索)。本演習は研修用のオリジナル問題であり、リンク先の問題・正解と同一ではありません。

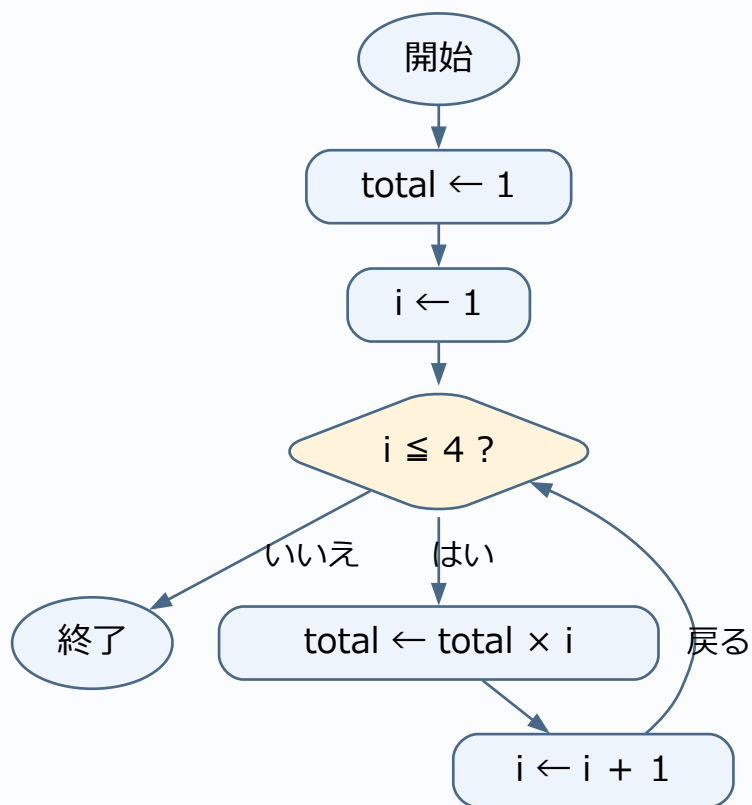
## 演習7-A 解答・解説

最初の誤りは $i=3$ の行です。 $2 \times 3 = 6$ なので、その行のtotalは6。次の行は $6 \times 4 = 24$ です。誤った5を使い続けな  
いよう、以降も直します。

値の確認(各処理の実行後)

| 実行した処理                      | その直後の値                 |
|-----------------------------|------------------------|
| $\text{total} \leftarrow 1$ | $\text{total}=1$       |
| $i=1$ の回を終了                 | $\text{total}=1, i=1$  |
| $i=2$ の回を終了                 | $\text{total}=2, i=2$  |
| $i=3$ の回を終了                 | $\text{total}=6, i=3$  |
| $i=4$ の回を終了                 | $\text{total}=24, i=4$ |

演習7-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

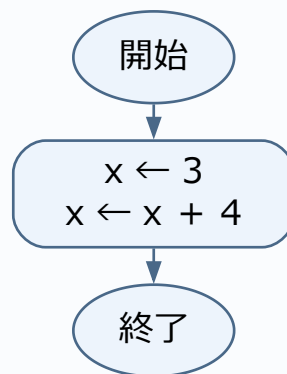
## 演習7-B 解答・解説

直前は3、直後は7です。右辺で使うxは、代入前の3です。

値の確認(各処理の実行後)

| 実行した処理               | その直後の値 |
|----------------------|--------|
| $x \leftarrow 3$     | $x=3$  |
| $x \leftarrow x + 4$ | $x=7$  |

演習7-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

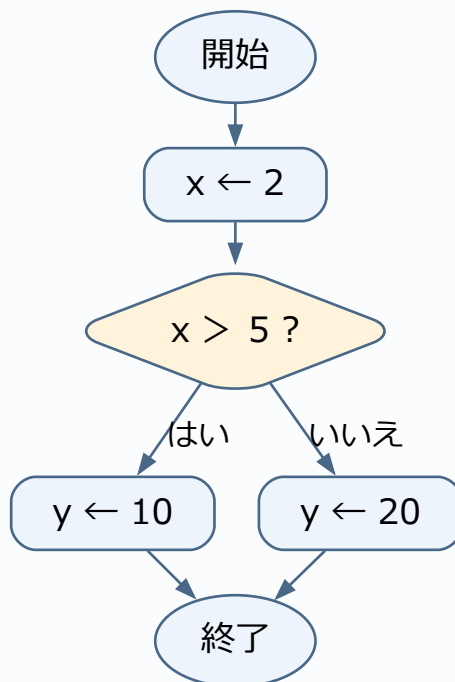
## 演習7-C 解答・解説

$x \leftarrow 2$ 、 $y \leftarrow 20$ が実行されます。 $2 > 5$ は偽なので $y \leftarrow 10$ は通りません。

値の確認(各処理の実行後)

| 実行した処理            | その直後の値          |
|-------------------|-----------------|
| $x \leftarrow 2$  | $x = 2$         |
| $y \leftarrow 20$ | $x = 2, y = 20$ |

演習7-Cのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

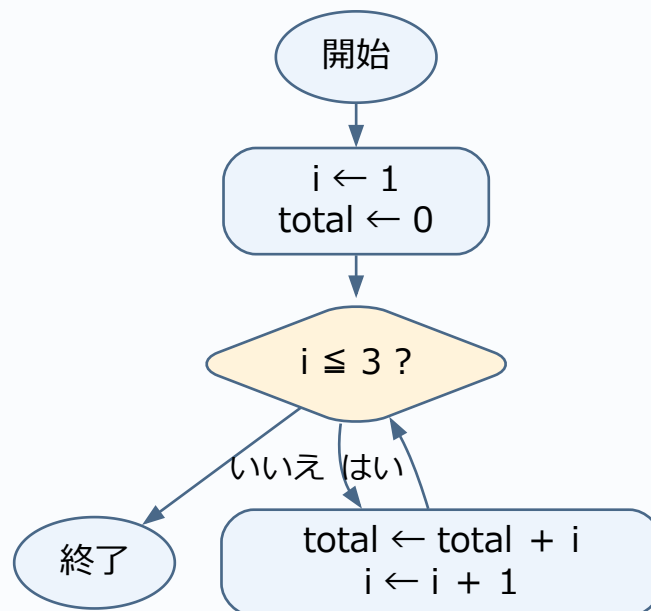
## 演習7-D 解答・解説

判定時の(i,真偽,total)は(1,真,0)、(2,真,1)、(3,真,3)、(4,偽,6)。終了時 $i=4$ 、 $total=6$ です。偽になった行も記録します。

値の確認(各処理の実行後)

| 実行した処理           | その直後の値            |
|------------------|-------------------|
| $i \leftarrow 1$ | $i=1$             |
| 繰返し一回分を終了        | $i=2$ 、 $total=1$ |
| 繰返し一回分を終了        | $i=3$ 、 $total=3$ |
| 繰返し一回分を終了        | $i=4$ 、 $total=6$ |

演習7-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。



## 演習7-E 解答・解説

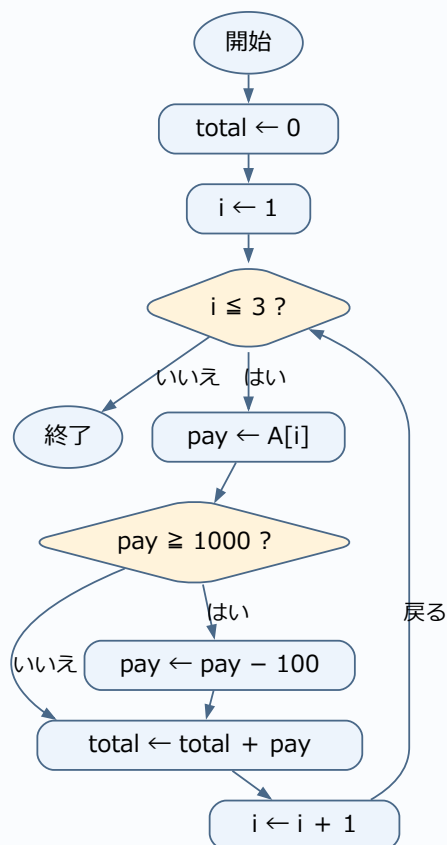
各回の(pay,total)は(999,999)、(900,1899)、(1400,3299)。>1000だと1000円の注文が値引きされず、合計は100円多い3399円です。

表の確認に使う入力:A={999,1000,1500}。

値の確認(各処理の実行後)

| 実行した処理    | その直後の値                  |
|-----------|-------------------------|
| total ← 0 | total=0                 |
| i=1の回を終了  | total=999、i=1、pay=999   |
| i=2の回を終了  | total=1899、i=2、pay=900  |
| i=3の回を終了  | total=3299、i=3、pay=1400 |

演習7-Eのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

## 演習7-F 解答・解説

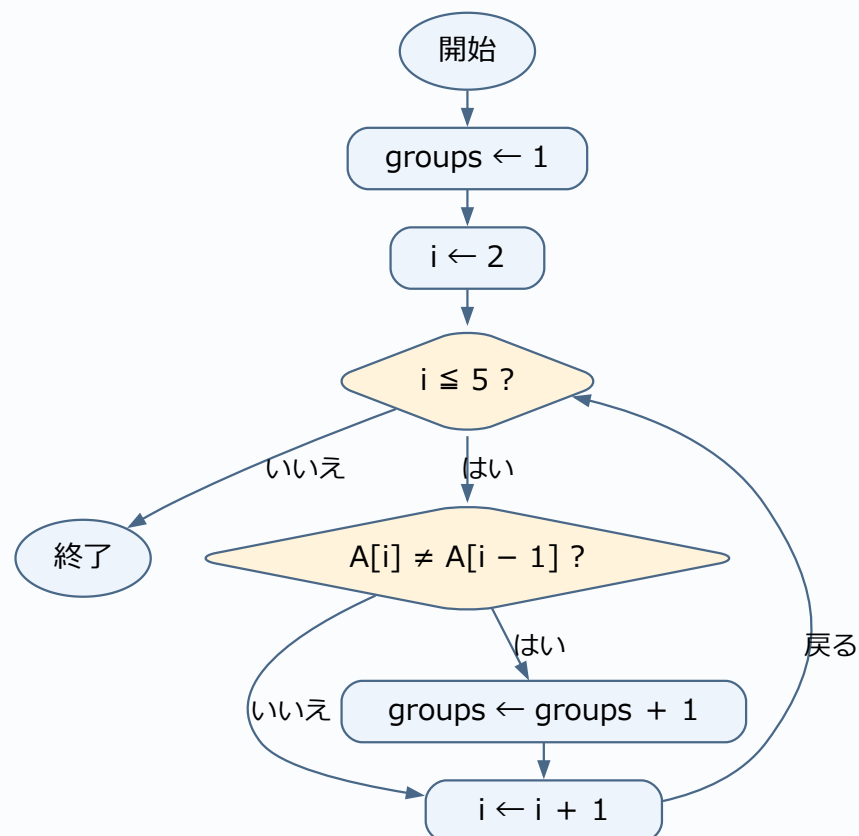
groupsは1→1→2→2→3で、答えは3まもりです。{2,2}、{5,5}、{2}と分かります。番号自体の種類は2種類なので、別の数え方です。

表の確認に使う入力: A = {2, 2, 5, 5, 2}。

値の確認(各処理の実行後)

| 実行した処理     | その直後の値            |
|------------|-------------------|
| groups ← 1 | groups = 1        |
| i = 2の回を終了 | groups = 1, i = 2 |
| i = 3の回を終了 | groups = 2, i = 3 |
| i = 4の回を終了 | groups = 2, i = 4 |
| i = 5の回を終了 | groups = 3, i = 5 |

演習7-Fのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

公開問題への接続: 配列を一行ずつ追う練習。本演習は研修用のオリジナル問題であり、リンク先の問題・正解と同一ではありません。

## 演習8-A 解答・解説

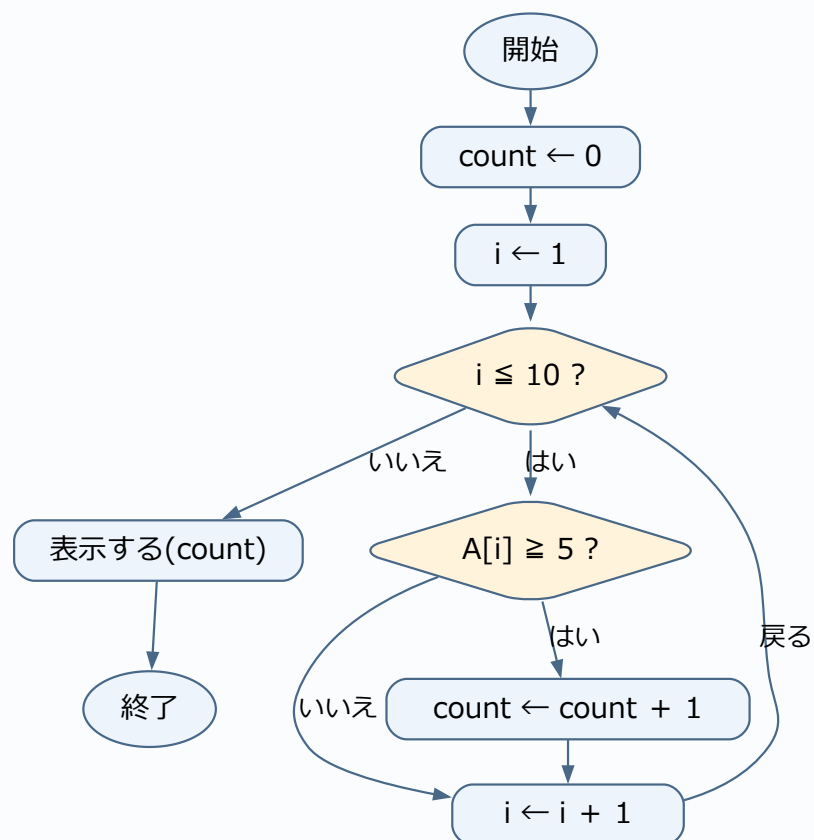
①0、② $A[i] \geq 5$ 、③ $\text{count} \leftarrow \text{count} + 1$ 。条件が真のときだけ1を足します。5自身も対象です。例えばAが1～10なら6件になります。

表の確認に使う入力:  $A = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ 。

値の確認(各処理の実行後)

| 実行した処理                      | その直後の値                     |
|-----------------------------|----------------------------|
| $\text{count} \leftarrow 0$ | $\text{count} = 0$         |
| i=1の回を終了                    | $\text{count} = 0, i = 1$  |
| i=2の回を終了                    | $\text{count} = 0, i = 2$  |
| ...(中間の反復を省略)               |                            |
| i=8の回を終了                    | $\text{count} = 4, i = 8$  |
| i=9の回を終了                    | $\text{count} = 5, i = 9$  |
| i=10の回を終了                   | $\text{count} = 6, i = 10$ |

演習8-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

## 演習8-B 解答・解説

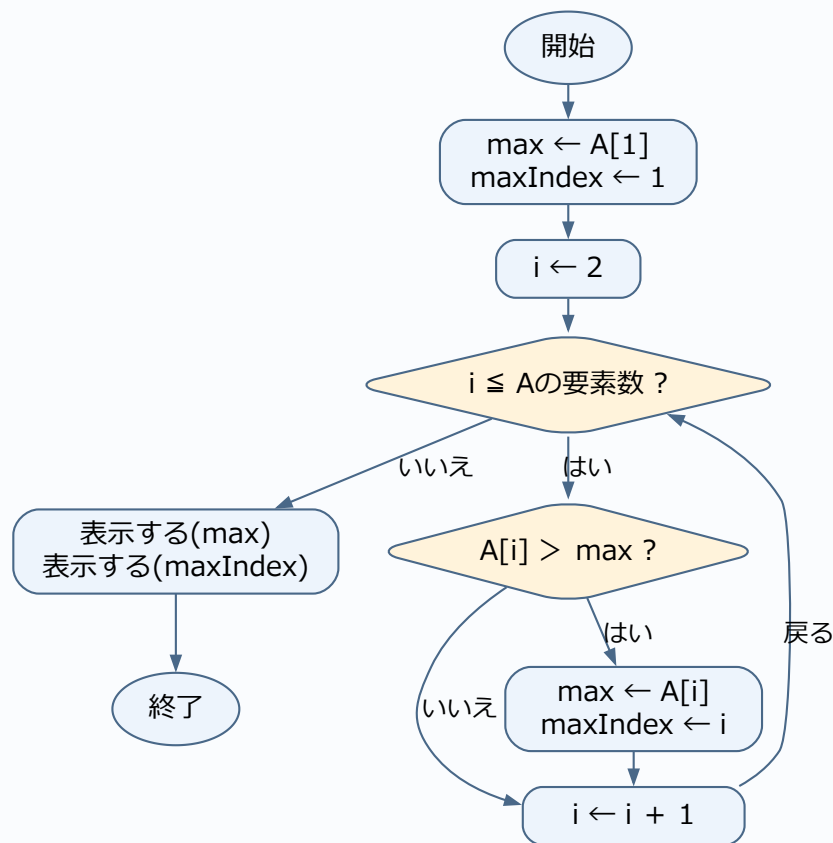
① $A[i] > \text{max}$ 、② $A[i]$ 、③ $i$ 。最大値と位置と一緒に更新します。 $>$ を使うため同点では更新されず、最初の位置が残ります。 $A = \{4, 9, 9\}$ なら $\text{max} = 9$ 、 $\text{maxIndex} = 2$ です。

表の確認に使う入力: $A = \{4, 9, 9\}$ 。

値の確認(各処理の実行後)

| 実行した処理                       | その直後の値                                             |
|------------------------------|----------------------------------------------------|
| $\text{max} \leftarrow A[1]$ | $\text{max} = 4$                                   |
| $i = 2$ の回を終了                | $\text{max} = 9$ 、 $\text{maxIndex} = 2$ 、 $i = 2$ |
| $i = 3$ の回を終了                | $\text{max} = 9$ 、 $\text{maxIndex} = 2$ 、 $i = 3$ |

演習8-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

公開問題への接続:配列の位置と上書きに注目する練習。本演習は研修用のオリジナル問題であり、リンク先の問題・正解と同一ではありません。

## 演習8-C 解答・解説

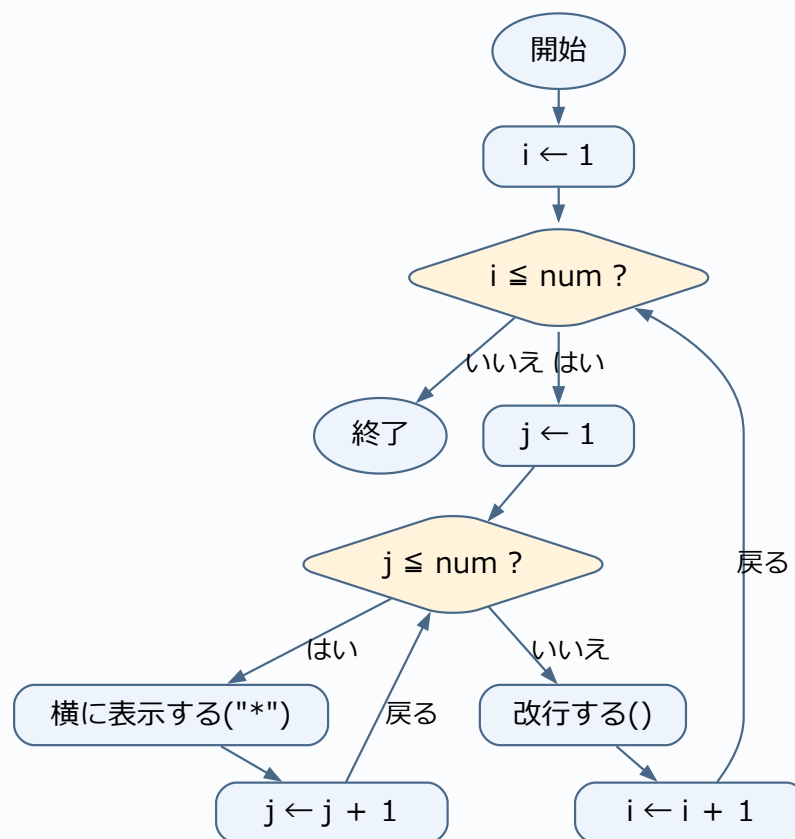
①  $i$  を 1 から  $num$  まで 1 ずつ増やす、②  $j$  を 1 から  $num$  まで 1 ずつ増やす、③ 横に表示する("\*")、④ 改行する()  
()。外側が行、内側が一行の文字を担当します。改行は内側を終えてから一回だけです。 $num=3$ なら、各行に \*\*\* を表示して3行になります。

表の確認に使う入力: $num=3$ 。

$num=3$ の表示結果を確認

| 外側の $i$ | 内側で表示する文字 | 内側を終えた後 |
|---------|-----------|---------|
| 1       | ***       | 改行する    |
| 2       | ***       | 改行する    |
| 3       | ***       | 改行する    |

演習8-Cのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

## 演習8-D 解答・解説

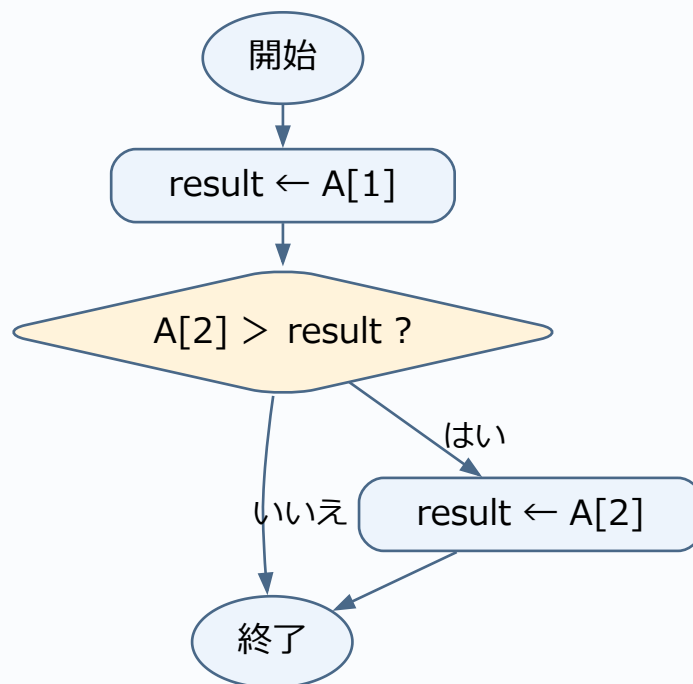
result=6です。初期値2よりA[2]の6が大きいので更新します。

表の確認に使う入力:A={2,6}。

値の確認(各処理の実行後)

| 実行した処理        | その直後の値   |
|---------------|----------|
| result ← A[1] | result=2 |
| result ← A[2] | result=6 |

演習8-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

## 演習8-E 解答・解説

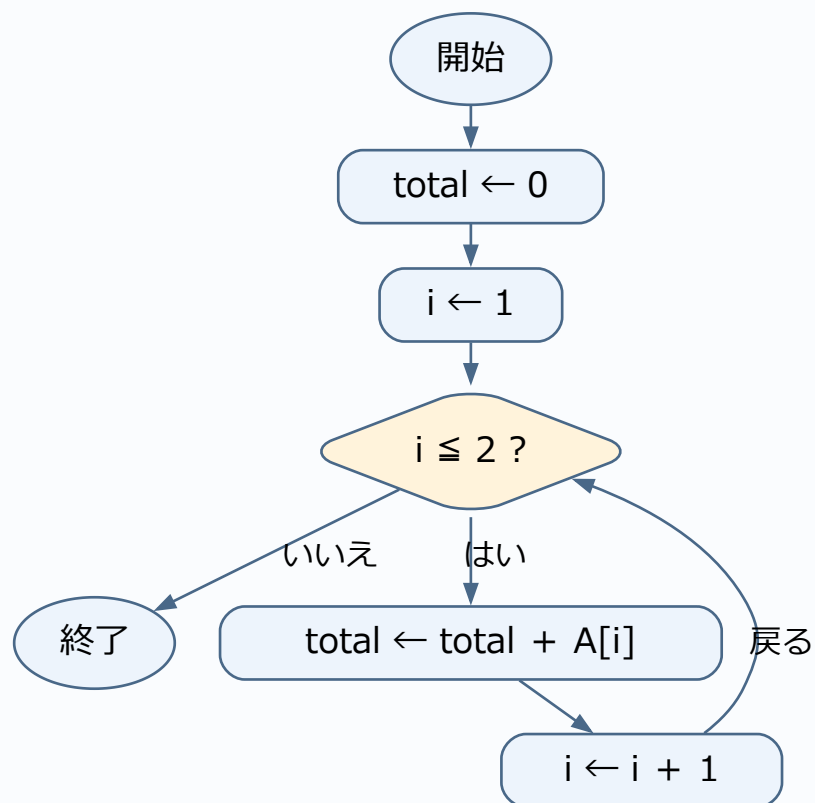
totalは0→3→8です。二つの要素を一回ずつ加算します。

表の確認に使う入力:  $A = \{3, 5\}$ 。

値の確認(各処理の実行後)

| 実行した処理               | その直後の値             |
|----------------------|--------------------|
| $total \leftarrow 0$ | $total = 0$        |
| $i = 1$ の回を終了        | $total = 3, i = 1$ |
| $i = 2$ の回を終了        | $total = 8, i = 2$ |

演習8-Eのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

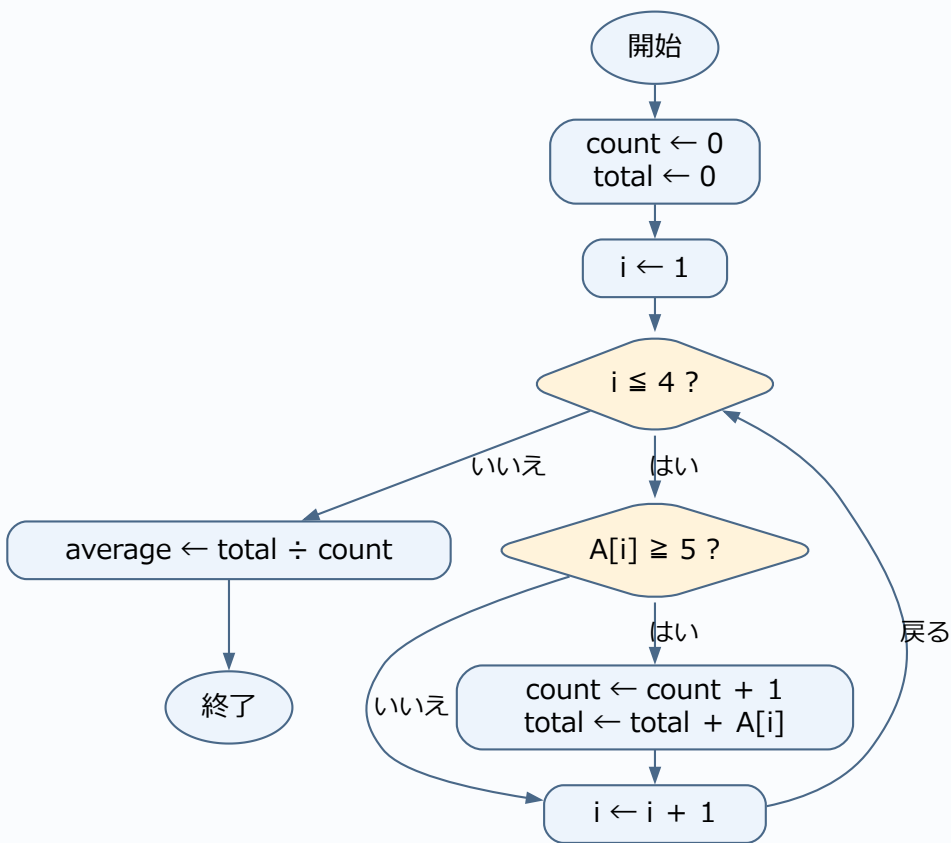
# 演習8-F 解答・解説

count=2、total=13、average=6.5。対象の8と5だけを足し、配列全体の4個ではなく対象の2個で割ります。  
表の確認に使う入力:A={3,8,5,1}。

値の確認(各処理の実行後)

| 実行した処理                  | その直後の値                           |
|-------------------------|----------------------------------|
| count ← 0               | count=0                          |
| i=1の回を終了                | count=0、total=0、i=1              |
| i=2の回を終了                | count=1、total=8、i=2              |
| i=3の回を終了                | count=2、total=13、i=3             |
| i=4の回を終了                | count=2、total=13、i=4             |
| average ← total ÷ count | count=2、total=13、i=5、average=6.5 |

演習8-Fのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。



# 研修後の学習ロードマップ

## 公開問題へ進む前の補助レッスン

公開問題は本教材より長く、まだ学んでいない言葉が登場することがあります。知らない単語だけで止まらず、次の表で読み方を確認します。解けなかった場合は、その知識を補ってから同じ問題へ戻しましょう。

| 新しい言葉・記法   | 初心者向けの意味                                             |
|------------|------------------------------------------------------|
| 関数         | 名前を付けてまとめた処理。例えば「合計を求める」という一まとまり。                    |
| 引数(ひきすう)   | 関数へ渡す材料。配列や、探したい値など。                                 |
| 戻り値・return | 処理の結果として呼出し元へ返す値。画面への表示とは別。returnに到達すると、その関数の処理を終える。 |
| 配列の末尾に追加   | 最後の後ろに新しい要素を一つ増やす。元の末尾の上書きとは違う。                      |
| 1ずつ減らすfor  | 例えばi=5、4、3、2の順で処理する。終了値2も処理する。                       |
| 昇順         | 小さい値から大きい値へ並んだ状態。                                    |
| 二分探索       | 並んだ配列の中央と比べ、探す範囲を狭める方法。単に先頭から探す方法とは違い、並び順が前提となる。     |

### 公開問題の読み解き例:配列の要素を移動する

令和8年度 科目B 問1と解説を開き、問題文の「末尾を先頭へ」「それ以外は一つ後ろへ」を確認してください。原問題と解答群はリンク先で読みます。以下は同じ処理の考え方を、小さい別の配列で練習するための説明です。

**先に確認:** len(レン)は要素数、top(トップ)は退避用の変数です。data[i - 1]は移動元、data[i]は移動先です。処理中に値を上書きするので、どちらを先に動かすかが重要です。

data={4,8,2,6}なら、目標は{6,4,8,2}です。まず最後の6をtopへ保存します。その後、iを4→3→2と減らしながら、一つ前の値を移します。

| 実行した処理            | data      | top |
|-------------------|-----------|-----|
| top ← data[4]     | {4,8,2,6} | 6   |
| data[4] ← data[3] | {4,8,2,2} | 6   |
| data[3] ← data[2] | {4,8,8,2} | 6   |
| data[2] ← data[1] | {4,4,8,2} | 6   |
| data[1] ← top     | {6,4,8,2} | 6   |

逆にiを2から増やすと、最初にdata[2]を4で上書きします。次に読むdata[2]も既に4なので、元の8を移せません。繰返しの方向は「増やすのが普通」と暗記せず、これから読む値を先に消していないかで判断します。原問題に戻り、開始値・終了値・増減の三つが合う選択肢を選びます。

## 配列が増える公開問題の読み方

サンプル問題 科目B 問3と解説では、配列の末尾への追加と、直前までに求めた値の利用を確認します。読む前に「引数が入力」「戻り値が出力」「末尾に追加すると要素数が1増える」を整理してください。

小さい別例で、入力が{2,4,1}なら、先頭からの合計を順に並べると{2,6,7}になります。2を置く→最後の2に4を足して6を追加→最後の6に1を足して7を追加、という順です。「入力配列」と「作成中の出力配列」を別々に表へ書けば、どの配列の最後を読んでいるか混同しにくくなります。

## 難しい探索問題は、前提を学んでから取り組む

サンプル問題 科目B 問13と解説は二分探索の不具合を扱います。第6章の「先頭から探す」練習より先の内容です。まず上の用語表を読み、lowは探索範囲の左端、highは右端、middleは中央の位置を表す変数だと整理します。

例えば二つの値{3,9}から9を探すとき、中央を小さい側の位置1とします。見つからなかったのに左端を再び1にすると、範囲は1～2のままです。同じ判定を繰り返して終わりません。「繰返した後に範囲が本当に小さくなったか」を、low・high・middleの表で確認します。講師は二分探索の前提を補足してから取り上げてください。

公開問題を自習するときの質問例：この公開問題の問題文・コードを提示します。まだ二分探索を習っていません。正解を先に示さず、必要な用語と前提を説明し、要素が二つの例で一回ずつ処理を確認してください。

これらの公開問題は共有PDFに原問題全文を収録していません。リンク先のページは、PDFにURLが書かれているだけではAIが読めるとは限りません。研修で扱う場合は講師が利用条件を確認してソースを準備し、自習では必要な問題文・コードを提示して読み取りを確かめます。本教材の演習1-A～8-Fは、共有PDF内の問題番号だけで質問できます。

本書で扱った内容は、科目Bのアルゴリズム問題を読むための入口です。次の順番で学習を続けます。

| 段階 | 学習内容            | 到達目標           |
|----|-----------------|----------------|
| 1  | 変数・代入・条件分岐・繰り返し | 短いコードをトレースできる  |
| 2  | 一次元・二次元配列       | 添字と要素を区別できる    |
| 3  | 合計・件数・最大・最小     | 基本パターンを説明できる   |
| 4  | 線形探索・二分探索       | 探索範囲の変化を追える    |
| 5  | 基本的な整列          | 要素の交換を追える      |
| 6  | スタック・キュー・リスト    | データ構造の操作を理解できる |
| 7  | 再帰・木・グラフ        | 呼出しと探索順を追える    |
| 8  | 科目B形式の総合問題      | 時間内に必要な値を追える   |

## 自習の一週間モデル

| 曜日  | 学習内容                        |
|-----|-----------------------------|
| 1日目 | 概念説明を読み、例題をトレースする           |
| 2日目 | 基本問題をAIなしで解く                |
| 3日目 | 間違えた問題についてAIへ質問する           |
| 4日目 | AIが作成した易しい類題を解く             |
| 5日目 | 本番形式の問題を時間を測って解く            |
| 6日目 | 間違いを「読解・条件・添字・計算・トレース」に分類する |
| 7日目 | AIを使わず、間違えた問題を解き直す          |

## 研修後に別のFE参考書で自習する場合

研修用の共有ノートブックは本教材の復習に使えます。別の参考書に進むときは、第0章0.4の方法で新しい問題をAIへ伝えてください。自分で利用権限のあるPDFを別のノートブックへ登録した場合も、教材名と問題番号を添えて対象を明確にします。

自習中の参考書『〇〇』第〇版、〇ページの問〇についてです。問題文とコードを添付します。まず入力条件と求める結果だけを確認してください。私は〇行目まで考えました。次の行で使う値が分からないので、答えではなく一つだけヒントをください。

添字の開始番号、配列の内容、関数の仕様、問題が複数ページにまたがる場合の続きも伝えます。AIが別の版や別の問題を説明したら、いったん止め、提供した内容だけで答えるように指定します。

生成AIへコピーして使える指示文

登録された教材を優先して回答してください。  
答えを直接示す前に、私がどこまで考えたかを確認してください。  
最初はヒントを一つだけ提示してください。  
擬似言語を説明するときは、変数の変化を表にしてください。  
私の考えに誤りがある場合は、最初の誤りだけを指摘してください。  
教材と異なる説明をする場合は、そのことを明示してください。

最後は必ずAIなしで解く

AIの説明を理解できても、本番でAIは使用できません。各単元の最後には、AIを閉じ、問題文と紙だけで同じ問題をもう一度解いてください。

学習記録

| 日付 | 学習範囲 | 自力で解けた問題 | AIへ質問した内容 | 解き直し                     |
|----|------|----------|-----------|--------------------------|
|    |      |          |           | <input type="checkbox"/> |
|    |      |          |           | <input type="checkbox"/> |
|    |      |          |           | <input type="checkbox"/> |
|    |      |          |           | <input type="checkbox"/> |

# 参考資料

---

- 基本情報技術者試験.com「科目B 過去問題」(公開問題の所在と解説を確認。教材の48問は研修用オリジナル問題。)
  - Google ヘルプ「ソースの追加」
  - Google ヘルプ「ノートブックの共有」
  - 独立行政法人情報処理推進機構(IPA)「試験要綱・シラバスについて」
  - 独立行政法人情報処理推進機構(IPA)「基本情報技術者試験 科目Bのサンプル問題」
  - 独立行政法人情報処理推進機構(IPA)「令和8年度 基本情報技術者試験 科目B 公開問題」
  - 独立行政法人情報処理推進機構(IPA)「SG・FE 公開問題」
- 

本教材の擬似言語は、基本情報技術者試験用の記述形式に合わせています。演習量、難易度、解説の粒度は、研修対象者と実施時間に合わせて調整します。